

SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Engineering



**DESIGN AND IMPLEMENTATION OF A REAL-TIME AUTOMATED
HYDROPONIC CULTIVATION SYSTEM USING IOT AND SENSORS.**

by

Mohamed Sheriff Mohamed Waseem

A thesis submitted in partial fulfillment of the requirements for the
Degree of Bachelor of Science of Engineering Honours in
Electrical and Electronic Engineering

May 2026

DECLARATION

I hereby declare that this thesis submitted to the Department of Electrical and Electronic Engineering, Faculty of Engineering, Sri Lanka Institute of Information Technology in partial fulfillment of the requirements for the award of the degree of Bachelor of Science of Engineering Honours in Electrical and Electronic Engineering is based on work done by me. The contents of this thesis have not been submitted by me to any other university or institute for the award of any other degree or diploma.

Signature :  _____

Name in Full : Mohamed Sheriff Mohamed Waseem

Registration No. : EN22155340

Date : 24-05-2026

Abstract

To achieve constant crop development in modern small-scale horticulture, precise monitoring and management of water circulation, nutrient supply, pH balance, and environmental variables are necessary. However, a lot of inexpensive hydroponic systems rely on human monitoring and offer nothing in the way of multi-node scalability, remote access, and real-time automation. In order to monitor and regulate the development conditions of tomatoes and chillies, this project created a real-time automated hydroponic cultivation system using IoT and sensors.

The Raspberry Pi 5-based local server, MQTT communication, server.js backend development, dashboard integration, mobile application support, network configuration, deployment, and remote access features were the main areas of my particular contribution. Using a Mosquitto MQTT broker, the Raspberry Pi 5 was set up as the central controller to receive JSON-based sensor data from several ESP32 boards. To process sensor readings, track ESP32 connectivity, carry out control logic, and interact with the web dashboard and mobile application, a Server.js backend server was created.

Separate ESP32 reading cards were used in the construction of the revised dashboard. Instead of using averaged readings, each ESP32 shows its own sensor values, last-seen status, control state, and actuator status. This enhanced system visibility and made it possible to monitor each ESP32 node separately. Water level management, pH-based nutrient dosing, and valve-pump operation were all managed by the automation. A relay module attached to an ESP32 GPIO 25 was used to regulate the nutrient dosing pump, which was triggered when the pH level rose above 6.5. Additionally, when the corresponding valve signals were recognised, the inlet and outlet pumps were automatically activated. The Raspberry Pi server, ESP32 nodes, MQTT connectivity, dashboard, relays, pumps, solenoid valves, and the mobile monitoring interface were all successfully integrated, according to testing.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, **Dr. Ruwan Chandrasena**, Head of the Department of Electrical and Electronic Engineering, for the guidance, support, and constructive feedback provided throughout this Final Year Project. His advice was valuable in shaping the design, development, and documentation of the Design and Implementation of a Real-Time Automated Hydroponic Cultivation System Using IoT and Sensors. I am also thankful to the module coordinator, **Dr. Lakmini Malasinghe**, for their support, coordination, and valuable suggestions during the project.

I would like to sincerely thank my group members, **Himas** and **Harindi**, for their commitment, teamwork, and technical contributions throughout the project. Their support in hardware development, embedded firmware, mobile application development, integration, and testing was essential for completing the system successfully. Finally, I thank everyone who directly or indirectly helped us during the completion of this project.

Table Of Contents

Declaration	ii
Abstract	iii
Acknowledgement	iv
Table Of Figure	ix
Table Of Tables	x
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement and Existing Gap	2
1.3 Proposed Solution and Scope of Individual Contribution	3
2 Literature review and technical overview	5
2.1 Hydroponic System Requirements	5
2.2 IoT and MQTT Communication in Monitoring Systems	5
2.3 Existing Gap in Low-Cost Hydroponic Automation	6
2.4 Dashboard and Mobile Monitoring Requirements	6
2.5 Summary of Technical Review	6
3 Design Methodology	8
3.1 Overview of the Design Approach	8
3.2 System Architecture Design	9
.....	9
3.3 Hardware Integration Methodology	10
3.3.1 Sensor and Actuator Integration	10
3.3.2 Pump and Solenoid Valve Coordination	11
3.4 MQTT Communication Methodology	11
3.4.1 MQTT Broker Setup	11
3.4.2 JSON Payload Processing	12
3.5 Backend Server Design Methodology	12
3.5.1 server.js Backend Development	12
3.5.2 Backend Communication with Dashboard and Mobile Application	13
3.6 Automated Control Logic Methodology	13
3.6.1 Water Level Control	13
3.6.2 Inlet and Outlet Pump Triggering Method	13

3.6.3	pH-Based Nutrient Dosing Control	14
3.6.4	TDS-Based Cycle Continuation	14
3.7	Web Dashboard Design Methodology	15
3.7.1	Separate ESP32 Reading Cards.....	15
3.7.2	Manual Control and Emergency Stop	16
3.8	Mobile Application Methodology	16
3.9	Network and Deployment Methodology	16
3.9.1	Static IP and Local Network Configuration	16
3.9.2	Wireless VNC Access.....	16
3.9.3	Docker-Based Deployment.....	17
3.9.4	Cloudflare and Domain-Based Remote Access.....	17
3.10	Testing and Validation Methodology.....	17
3.10.1	Sensor and MQTT Testing.....	17
3.10.2	Relay and Actuator Testing.....	17
3.10.3	Dashboard and Mobile Application Testing	17
3.10.4	Full Automation Cycle Testing	18
3.11	Summary of Design Methodology	18
4	Implementation	19
4.1	Implementation Environment	20
4.1.1	Hardware Implementation Environment	20
4.1.2	Software Implementation Environment	21
4.2	ESP32 Sensor Node Implementation	21
4.2.1	Sensor Interfacing.....	22
4.2.2	JSON Data Formatting.....	23
4.2.3	MQTT Data Publishing	23
4.3	Raspberry Pi 5 Local Server Implementation	24
4.3.1	Raspberry Pi Operating System Configuration	25
4.3.2	Mosquitto MQTT Broker Installation and Configuration	25
4.3.3	Network Configuration	25
4.4	Server.js Backend Server Implementation	25
4.4.1	MQTT Subscription Handling	26
4.4.2	Sensor Data Processing	26
4.4.3	API and Dashboard Communication	26

4.5	Web Dashboard Implementation	26
4.5.1	Real-Time Monitoring Interface	27
4.5.2	Manual Control Interface.....	27
4.5.3	ESP32 Category-Based Display.....	27
4.5.4	Mobile Responsive Interface.....	28
4.6	Actuator and Relay Control Implementation	28
4.6.1	Inlet and Outlet Valve Control	28
4.6.2	Nutrient Dosing Pump Control	28
4.7	Automatic Control Logic Implementation	29
4.7.1	Water Level Control	29
4.7.2	pH Monitoring and Alert Logic	30
4.7.3	TDS-Based Control	30
4.7.4	Drain and Refill Cycle	30
4.8	Email Alert and Remote Monitoring Implementation	31
4.9	System Deployment	31
4.10	Chapter Summary	32
5	Integration and testing.....	33
5.1	System Integration Overview.....	33
5.2	Hardware Integration.....	35
5.3	Software and Communication Integration.....	36
5.4	Dashboard and Mobile Interface Testing	38
5.5	Actuator and Relay Testing.....	40
5.6	Automatic Control Logic Testing.....	41
5.7	Network and Remote Monitoring Testing	41
5.8	System-Level Testing and Results.....	42
5.9	Problems Encountered and Corrective Actions	42
5.10	Chapter Summary	43
6	RESULTS AND DISCUSSION.....	44
6.1	Introduction.....	44
6.2	Sensor Reading Results	44
6.3	MQTT Communication Results.....	45
6.4	Web Dashboard and Mobile Interface Results	46
6.5	Actuator Control Results	46

6.6 Automatic Control Logic Results	47
6.7 System-Level Performance Discussion	48
6.8 Comparison with Expected Results	49
6.9 Limitations Observed	49
6.10 Summary of Results	50
7 conclusion & Discussion	52
References	57
Appendix a	60
Appendix b	62

Table Of Figure

Figure 1.1 Overall block diagram of the automated hydroponic cultivation system	4
Figure 3.1 Layered architecture of the proposed IoT-based hydroponic system	9
Figure 3.2 Web dashboard with separate ESP32 reading cards	15
Figure 4.1 Overall implementation workflow of the hydroponic system	19
Figure 4.2 ESP32 sensor node implementation diagram.....	22
Figure 4.3 MQTT communication flow between ESP32, Raspberry Pi, backend, and dashboard.....	23
Figure 4.4 Raspberry Pi 5 local server deployment structure.....	24
Figure 4.5 Web dashboard and mobile interface implementation layout	27
Figure 4.6 Automatic hydroponic control logic flowchart.....	29
Figure 5.1 Overall system integration diagram	33
Figure 5.3 Hardware and software integration workflow	35
Figure 5.3 MQTT data communication testing sequence	36
Figure 5.4 Dashboard-to-actuator manual control test flow	38

Table Of Tables

Table 2.1 Comparison between existing hydroponic systems and the proposed system	7
Table 3.1 Hardware and software components used in the system	9
Table 3.2 ESP32 pin allocation for sensors and actuators	11
Table 3.3 MQTT topic structure used in the system	12
Table 3.4 Control threshold values used in the automation logic.....	14
Table 4.1 Hardware components used in the implementation	20
Table 4.2 Software tools and technologies used	21
Table 4.3 ESP32 sensor and actuator pin configuration	22
Table 4.4 MQTT topic structure and JSON payload description	24
Table 4.5 Main control conditions used in the automation logic	31
Table 5.1 Integrated system components and their functions	34
Table 5.2 MQTT communication and data verification test cases	37
Table 5.3 Dashboard and mobile interface test cases	39

1 INTRODUCTION

1.1 Background and Motivation

In order to achieve constant crop development and output, modern small-scale horticulture increasingly needs precise control over nutrient supply, pH balance, water circulation, and environmental variables. Using a nutrient-rich water solution in place of soil, hydroponic gardening is a soilless farming technique. Compared to conventional soil-based agriculture, this technology allows for more effective management of water, nutrients, and environmental conditions, making it appropriate for controlled development of crops like tomatoes and chillies.

However, a hydroponic system's ability to function depends on the ongoing maintenance of critical variables such temperature, humidity, pH, water level, and total dissolved solids. Plant growth may be impacted by nutrient imbalance, inappropriate pH levels, inadequate water circulation, or unstable environmental circumstances if these factors are not kept within appropriate ranges. Thus, automated control and real-time monitoring are crucial components of a workable hydroponic farming system.

Small farmers still rely on manual monitoring and control for a large number of inexpensive hydroponic systems. In these systems, the user must manually operate pumps, valves, or dosing mechanisms after periodically checking the water level, nutrient content, and pH condition. In addition to requiring more human labour, this strategy might postpone remedial measures. For instance, the system might not react right away if the pH value shifts outside of the necessary range or the nutritional solution level drops, unless the user notices the change. This necessitates the development of an automated system that can regulate the necessary actuators with the least amount of human intervention and monitor hydroponic conditions in real time.

The goal of this project was to develop and put into use an inexpensive, Internet of Things-based automated hydroponic gardening system for the production of two varieties of tomatoes and chillies. The system was built to automatically regulate solenoid valves, pumps, and a nutrient dosing pump while monitoring water level, pH, TDS, temperature, and humidity. Through a web dashboard, mobile application, local server control, and remote monitoring, the project also sought to increase user accessibility.

1.2 Problem Statement and Existing Gap

The absence of an affordable, scalable hydroponic automation system that integrates real-time sensor monitoring, automated actuator control, local server processing, dashboard visualisation, mobile access, and remote monitoring is the primary issue this project attempts to solve. While there are a number of hydroponic monitoring options, many inexpensive systems still require manual control of pumps, valves, and nutrient dosing mechanisms and mostly concentrate on showing sensor information. This makes it more difficult for them to sustain steady growth conditions without constant user oversight.

The division of pumps and water flow management is another drawback of basic hydroponic systems. Solenoid valves are only utilised in some systems to open or close water flow pathways; gravity or independent manual pump control are required to actually transfer the water. Particularly in small-scale practical systems where water tanks, pipes, and plant beds are positioned at different levels, this approach might not offer dependable filling and draining performance. Therefore, to increase the reliability of water circulation, a coordinated valve and pump control approach is needed.

Furthermore, a lot of small-scale systems lack centralised control and robust multi-node communication. The system needs a reliable way for ESP32 boards and a central controller to communicate when many sensors and actuators are employed. Processing real-time data, monitoring device status, applying control logic, and continuously updating the dashboard become challenging without an appropriate backend architecture. As a result, the project needed a centralised Raspberry Pi-based control system that could collect sensor data, process it, and regulate the hydroponic system in an organised manner.

By utilising ESP32 boards, Raspberry Pi 5, Mosquitto MQTT broker, Server.js backend server, web dashboard, mobile application, relays, solenoid valves, input and outlet pumps, and a nutrient dosing pump, an Internet of Things-based hydroponic growth system was developed to fill the identified gap. In addition to monitoring sensor data, the system was built to automatically control valve-pump operation, pH-based nutrition dosing, and water level adjustment.

1.3 Proposed Solution and Scope of Individual Contribution

The proposed system consists of three ESP32 boards, a Raspberry Pi 5 local server, Mosquitto MQTT broker, Server.js backend server, web dashboard, mobile application, sensors, relay modules, solenoid valves, water pumps, and a nutrient dosing pump. The ESP32 boards collect real-time readings from the pH sensor, TDS sensor, ultrasonic sensor, and DHT11 temperature and humidity sensor. These readings are transmitted to the Raspberry Pi through MQTT communication using JSON payloads. The Raspberry Pi 5 acts as the central local server and main controller by processing the received data and executing the required control logic.

The automated procedure first uses feedback from an ultrasonic sensor to maintain the necessary water level. The inflow solenoid valve is triggered when the water level falls below the necessary level. When a control signal reaches the inlet valve in the revised design, the inlet pump also starts up automatically. Every pump uses an ESP32-based signal scanning technique, in which the associated pump relay pin is triggered once the ESP32 scans the relevant valve signal pin. As a result, the inlet pump relay is automatically activated when any inlet valve receives a signal. Similarly, the outlet pump relay is automatically activated upon receiving a signal from any outlet valve. Because the pump and valve work simultaneously, filling and draining activities are more reliable.

The mechanism measures the pH level once the water level reaches the necessary amount. The ideal pH level is roughly 7. A relay module attached to an ESP32 GPIO 25 activates the nutrition dosing pump if the pH level is higher than 6.5. To prevent overdosing, the dosing pump runs for a predetermined amount of time. The solution is given time to mix after dosage before the readings are reassessed. The system measures the TDS value when the pH level stabilises. The outlet valve and outlet pump work to drain the solution until the necessary drain level is attained after the necessary TDS condition is met. After then, the cycle automatically repeats.

For real-time monitoring and control, the system also has a mobile application and web dashboard. Sensor data, valve status, pump status, nutrition dosing pump status, ESP32 device status, and system log messages are all shown on the dashboard. Inlet and outlet valve control, pump control, nutrition pump control, auto/manual mode selection, and emergency stop are additional manual control features. These features facilitate maintenance, testing, troubleshooting, and demonstration.

The Raspberry Pi 5 server and controller, Mosquitto MQTT broker setup, Server.js backend development, control logic implementation, web dashboard integration, mobile application support, Docker deployment, static IP configuration, wireless VNC access, and remote access via domain and Cloudflare services were the main areas of focus for me as Member 2. Additionally, I collaborated with Member 1 on system testing, actuator integration, and hardware wiring. Thus, the backend control, communication, deployment, remote access, and integration components of the entire automated hydroponic growth system are the primary emphasis of this paper.

The creation of a distinct ESP32 monitoring interface in the online dashboard was a significant improvement to the suggested system. Each ESP32 was shown as a separate reading card rather than aggregating all ESP32 sensor readings into a single averaged result. The device name, online/offline status, last-seen status, sensor readings, inlet and outlet valve status, auto/manual mode, emergency stop option, and system logic state are all displayed on each card. Because the user can determine whether ESP32 node is active, disconnected, or awaiting telemetry, this architecture enhances fault discovery. Because more ESP32 nodes may be monitored independently without influencing the readings of other devices, it also facilitates improved system scalability.

Figure 1.1 depicts the general idea of the suggested Internet of Things-based automated hydroponic farming system. The ESP32 sensor nodes, MQTT communication, Raspberry Pi 5 backend controller, web dashboard, mobile application, actuator system, and remote access services are all summarised in the figure.

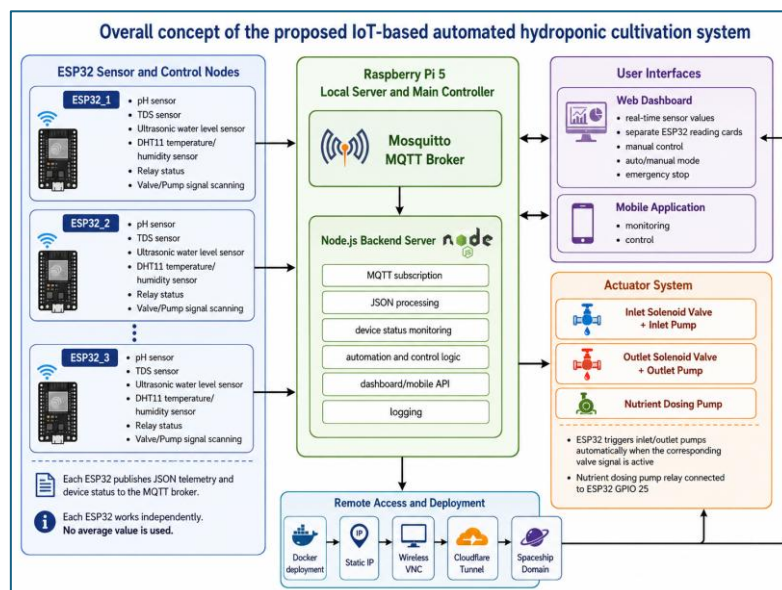


Figure 1.1 Overall block diagram of the automated hydroponic cultivation system

2 LITERATURE REVIEW AND TECHNICAL OVERVIEW

2.1 Hydroponic System Requirements

Stable control of the surrounding environment and the nutrient solution is necessary for a hydroponic cultivation system. The water level, pH, total dissolved solids, temperature, and humidity are the most crucial factors for this project. While pH and TDS monitoring are necessary to maintain an appropriate chemical condition for nutrient uptake, water level adjustment is necessary to keep the nutrient solution accessible to the plant roots. Additional environmental data that helps users comprehend the growing conditions within the crop area is provided by temperature and humidity readings.

Additionally, the system needs to be practical to maintain, inexpensive, and easy to monitor for small-scale farmers. When the user anticipates automatic water filling, draining, and nutrition dosing, a system that merely shows measurements is insufficient. Thus, sensor monitoring, actuator control, local decision-making, dashboard visualisation, and mobile access were all necessary for the proposed project.

2.2 IoT and MQTT Communication in Monitoring Systems

Distributed microcontroller nodes are frequently used in Internet of Things-based systems to gather sensor data and send it to a central server. The Raspberry Pi 5 served as the central local server in this project, and the ESP32 boards served as sensor and actuator nodes. This method made it possible for the Raspberry Pi to carry out higher-level control, dashboard communication, deployment, and remote access tasks while lessening the processing load on the ESP32 boards.

Because MQTT communication allows for lightweight publish/subscribe communications, it was chosen for the suggested system. The Mosquitto MQTT broker on the Raspberry Pi can receive telemetry, status messages, and logs from each ESP32. Subscribing to those topics, processing JSON payloads, updating the dashboard, and sending commands back to the ESP32 nodes are all possible with the Server.js backend. Because each ESP32 may function independently while yet being controlled by the central backend, this communication structure is appropriate for a multi-node hydroponic system.

2.3 Existing Gap in Low-Cost Hydroponic Automation

Many inexpensive hydroponic systems can only do simple sensing and display tasks. Although these systems may display pH, water level, or nutritional information, they frequently lack a full automation sequence for filling, dosing, emptying, refilling, manual override, and emergency stop. Monitoring and actual control become practically separated as a result. The absence of transparent multi-node monitoring is another shortcoming. Combining measurements from multiple ESP32 boards into a single average number can conceal issues like a malfunctioning sensor, a disconnected node, or an unusual reading from a single crop area. Because of this, distinct ESP32 reading cards were used in the design of the suggested system to enable independent monitoring of each board.

2.4 Dashboard and Mobile Monitoring Requirements

A user interface that clearly shows sensor readings, actuator status, controller state, and device connectivity is necessary for a workable hydroponic monitoring system. While the mobile application offers a more practical interface for viewing one ESP32 unit at a time, the web dashboard offers real-time monitoring and manual control from a browser. These interfaces facilitate testing, debugging, and operation while also enhancing accessibility.

The mobile application was created to use the configured remote access path to connect to the current Raspberry Pi backend API. Server login, ESP32-based navigation, analog-style sensor gauges, actuator status, manual control, emergency stop, and controller reset are all supported. The most recent mobile application version solves toggling between ESP32 units and only displays ESP32 1, ESP32 2, and ESP32 3 in the sidebar. Choosing ESP32 2 or ESP32 3 displays the right board rather than ESP32 1 continuously.

2.5 Summary of Technical Review

Real-time sensing, robust connectivity, centralised backend control, node-level monitoring, valve-pump coordination, dashboard visualisation, mobile access, and remote deployment are all necessary components of the suggested hydroponic system, according to the technical review. Table 2.1 compares the suggested system with popular, inexpensive hydroponic systems currently in use.

Table 2.1 Comparison between existing hydroponic systems and the proposed system

Existing System Feature	Common Limitation	Proposed System Improvement
Sensor monitoring	Only displays readings	Real-time dashboard and mobile monitoring
Manual pump/valve control	Requires user action	Automated valve and pump control
Single controller system	Limited scalability	Multiple ESP32 node support
Basic dashboard	Combined or averaged readings	Separate ESP32 reading cards
Local-only access	Cannot monitor remotely	Cloudflare/domain remote access

3 DESIGN METHODOLOGY

3.1 Overview of the Design Approach

This project's design methodology was created to use Internet of Things-based monitoring and control to establish a real-time automated hydroponic production system. In addition to controlling actuators like intake solenoid valves, outlet solenoid valves, inlet pumps, outlet pumps, and the nutrient dosing pump, the complete system was built to monitor critical hydroponic parameters including water level, pH, total dissolved solids, temperature, and humidity.

Three primary functioning groups comprised the system. ESP32 boards, sensors, relay modules, solenoid valves, and pumps comprised the initial group, known as the sensing and actuator group. The second group, known as the communication and control group, employed MQTT connectivity between the Raspberry Pi 5 and the ESP32 boards. The web dashboard, mobile application, Docker deployment, static IP configuration, wireless VNC access, and Cloudflare/domain-based remote monitoring comprised the third group.

As Member 2, I primarily concentrated on the system's central server, backend processing, MQTT communication, automated control logic, dashboard integration, support for mobile applications, deployment, and remote accessibility. During the hardware implementation and testing phases, Member 1 and I collaborated as well.

Table 3.1 lists the primary hardware and software components utilised in the suggested system. Sensor monitoring, MQTT communication, backend processing, actuator control, dashboard visualisation, mobile access, and remote deployment are all supported by these components.

Table 3.1 Hardware and software components used in the system

Category	Component / Software	Purpose
Controller	Raspberry Pi 5	Local server and main controller
Microcontroller	ESP32 boards	Sensor reading and actuator control
Communication	Mosquitto MQTT	Real-time data transfer between ESP32 nodes and Raspberry Pi
Backend	Server.js	Data processing, device monitoring, and control logic
Interface	Web dashboard	Real-time monitoring and manual control
Interface	Mobile application	ESP32-based monitoring and control from a mobile device
Deployment	Docker	Backend service packaging and management
Remote access	Cloudflare tunnel and domain service	External dashboard and API access

3.2 System Architecture Design

As seen in Figure 3.1, the system architecture was created as a layered Internet of Things structure. The sensor layer, ESP32 node layer, MQTT communication layer, Raspberry Pi backend layer, dashboard/mobile layer, and actuator layer are among the layers. While control commands travel downward from the Raspberry Pi backend to the actuators via the ESP32 nodes, telemetry and status data travel upward from the sensors to the user interfaces.

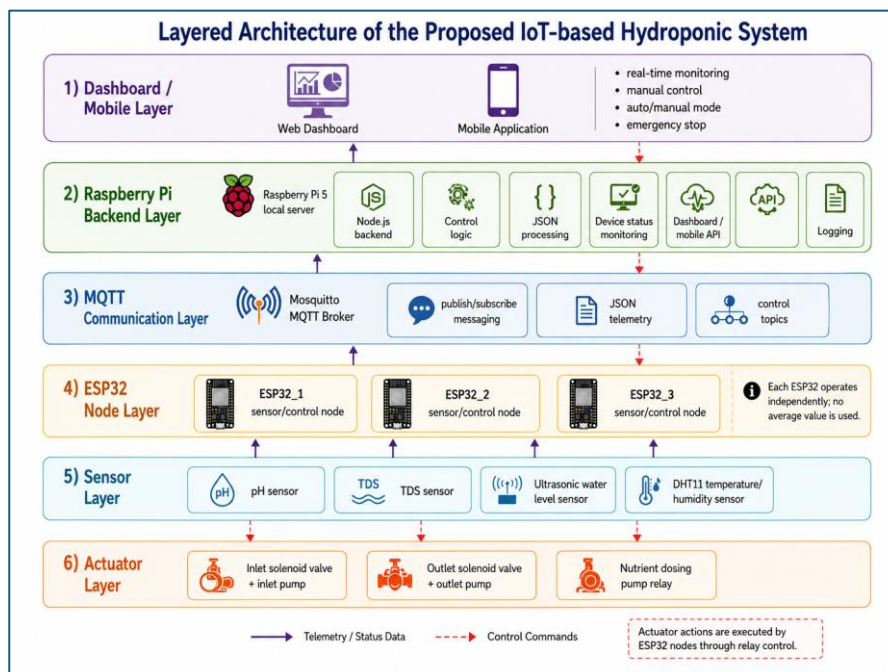


Figure 3.1 Layered architecture of the proposed IoT-based hydroponic system

3.2.1 Distributed ESP32 Node Architecture

Instead of utilising a single microcontroller for all sensing and actuator duties, the system was developed with several ESP32 boards. This method allowed for the separation of sensor readings and control states for every ESP32 node and enhanced system scalability. Each ESP32 was in charge of gathering sensor data or managing actuator circuits that were connected, then transmitting that data to the Raspberry Pi server.

Separate ESP32 reading cards were employed in the revised dashboard design. Temperature, humidity, pH, TDS, water distance, Wi-Fi RSSI, device status, last-seen time, control mode, and actuator state were all shown individually by each ESP32 card. Between ESP32 boards, no average value was utilised. This technique made it possible to monitor each ESP32 node separately and assisted in locating any issues with communication, missing telemetry, or unusual sensor data from a particular node.

3.2.2 Raspberry Pi 5 Central Control Architecture

Because it can run a complete operating system, host the MQTT broker, run a Server.js backend server, facilitate dashboard communication, and control local or distant access, the Raspberry Pi 5 was chosen as the central local server and control unit. The backend control procedure was supported by the installation and configuration of Raspberry Pi OS.

After receiving MQTT data from the ESP32 boards, the Raspberry Pi evaluated the incoming JSON payloads, updated the mobile application and dashboard, and carried out the automated control logic. High-level decision-making and low-level sensor reading were kept apart in this approach. Consequently, the Raspberry Pi handled central processing and system coordination, whereas ESP32 boards handled local input/output.

3.3 Hardware Integration Methodology

3.3.1 Sensor and Actuator Integration

Sensors for pH, TDS, water level, temperature, and humidity were incorporated into the hardware design. The status of the nutrition solution was tracked using the pH and TDS sensors. The solution level was ascertained by measuring the water distance from the ultrasonic sensor. The growing environment's temperature and humidity were measured using the DHT11 sensor.

The nutrition dosing pump, inlet pump, outlet pump, and inlet and outlet solenoid valves were all controlled by relay modules. Higher-power actuator circuits could be switched by the ESP32 control signals thanks to the relay modules. I helped Member 1 wire and test the ESP32 boards, sensors, relays, solenoid valves, and pumps throughout implementation. Table 3.2 lists the ESP32 pin allocation utilised for the actuators and sensors.

Table 3.2 ESP32 pin allocation for sensors and actuators

Device	ESP32 GPIO Pin	Function
Inlet solenoid valve relay	GPIO 27	Inlet valve control
Outlet solenoid valve relay	GPIO 26	Outlet valve control
Nutrient dosing pump relay	GPIO 25	Nutrient pump control
TDS sensor	GPIO 35	TDS measurement
pH sensor	GPIO 34	pH measurement
Ultrasonic sensor Trig	GPIO 5	Trigger signal
Ultrasonic sensor Echo	GPIO 18	Echo signal
DHT11 sensor	GPIO 4	Temperature and humidity measurement

3.3.2 Pump and Solenoid Valve Coordination

A change in design was made such that the water pumps start working automatically based on the state of their corresponding solenoid valves. Upon sensing that an inlet valve receives an active control signal, the ESP32 checks the inlet valve signal pin and activates the inlet pump relay. Likewise, upon sensing that an outlet valve receives an active control signal, the ESP32 checks the outlet valve signal pin and activates the outlet pump relay.

In this way, the coordination between the valve and the pump becomes better. The inlet pump will be activated only if there is an active inlet valve, and the outlet pump will be activated only if there is an active outlet valve.

3.4 MQTT Communication Methodology

3.4.1 MQTT Broker Setup

The Mosquitto MQTT broker was set up on the Raspberry Pi 5 device. The MQTT broker was utilized as a means of communication between the ESP32 boards and the Server.js server. All the ESP32 sensors sent their values through the MQTT broker to particular topics.

MQTT was chosen due to its efficiency and simplicity of implementation; it is also ideal for IoT scenarios where communication needs to be established with multiple devices. It provided a way of subscribing to several ESP32 data streams in real-time from the Raspberry Pi back-end. The MQTT topic naming for telemetry, status, logging, commands, and control publications is described in Table 3.3 below.

Table 3.3 MQTT topic structure used in the system

Purpose	Topic Pattern	Direction
Sensor readings	hydroponics/<device_id>/telemetry	ESP32 to Raspberry Pi
Serial logs as JSON	hydroponics/<device_id>/logs	ESP32 to Raspberry Pi
Online/offline status	hydroponics/<device_id>/status	ESP32 to Raspberry Pi
Relay/manual/emergency command	hydroponics/<device_id>/command	Raspberry Pi to ESP32
Controller state	hydroponics/<device_id>/controller	Raspberry Pi to MQTT retained

3.4.2 JSON Payload Processing

ESP32 boards were sending the collected data in the JSON format. The payload had structured data which included device ID, data from the sensors, actuator status, strength of the Wi-Fi connection, and system status among others. The backend was receiving the collected data from the MQTT messages after parsing the JSON data and splitting the values for each ESP32 node.

The approach enabled the backend to send data to the right dashboard cards corresponding to the particular ESP32 nodes. No averaging of data was done hence making it easy to detect faults and making the dashboards more useful for hydroponics.

3.5 Backend Server Design Methodology

3.5.1 server.js Backend Development

The backend server was created using server.js. The functions of the backend were MQTT topic subscription, sensor data analysis, actuator state control, monitoring the state of the device, communicating with the dashboard, communicating with the mobile application, and executing the control algorithm.

The backend was tracking the current readings of each ESP32 and kept the interfaces up to date. Also, it kept track of the last seen time for each ESP32. In case when ESP32 failed to provide any readings, the dashboard showed it as disconnected or waiting for telemetry.

3.5.2 Backend Communication with Dashboard and Mobile Application

The backend using Server.js was responsible for acting as an interface between the Raspberry Pi controller system, web dashboard, and mobile application. The readings of the sensors and status of actuators were transferred from the backend to the web dashboard and the mobile application. The backend system would also receive control instructions from the web dashboard and mobile application to the appropriate control function.

Such an approach would provide automatic control as well as manual control options. In the case of automatic control, control was executed on the basis of the value sensed by the Raspberry Pi backend. The manual control included the ability to remotely control the inlet valve, outlet valve, nutrient pump, inlet pump, outlet pump, and emergency stop function.

3.6 Automated Control Logic Methodology

3.6.1 Water Level Control

Water level control was the first process to be carried out in the automation chain. It used an ultrasonic sensor to measure the distance from the sensor to the water level. The desired water level distance was kept at about 3 cm from the ultrasonic sensor.

If the distance reading was more than 3 cm, it meant that the water level was low, hence the inlet valve was ON while the outlet valve was OFF. When the water level rose to the desired level, the inlet valve was turned OFF.

3.6.2 Inlet and Outlet Pump Triggering Method

ESP32 signal scanning was done for the inlet pump and the outlet pumps. In the event that the inlet valve signal would become active, then the ESP32 would read the signal and trigger the inlet pump relay. In the event that the outlet valve signal would become active, then the ESP32 would read the signal and trigger the outlet pump relay.

This helped synchronize valve and pump functions, whereby the inlet pump was used in the process of filling and the outlet pump was used in the process of draining.

3.6.3 pH-Based Nutrient Dosing Control

Once the water level met the required criteria, the system proceeded to test the pH value. The desired pH value was 7. In case the pH level was more than 6.5, then the nutrient dosing pump would be turned on using the relay module which would get instructions from ESP32 GPIO 25.

The system would keep the nutrient dosing pump running for an optimal period of time to avoid overdosing. Once the nutrient dosing process ended, the system would wait until the nutrient mixture was mixed well before taking any other measurements.

3.6.4 TDS-Based Cycle Continuation

Once the pH stability was achieved, the system tested the TDS level. The targeted TDS level was about 1500 ppm. Once the TDS level attained the required condition, the outlet valve opened while the inlet valve stayed OFF.

At this point, the outlet pump was turned on automatically by receiving an output from the outlet valve. The solution was drained till it reached a distance of about 6 cm to 7 cm. At this stage, the output valve and output pump were turned OFF, followed by turning ON the input valve and input pump to refill the solution to a distance of about 3 cm. This is illustrated by the threshold levels shown in Table 3.4.

Table 3.4 Control threshold values used in the automation logic

Parameter	Threshold / Target Value	Control Action
Water distance	Greater than 3 cm	Turn ON inlet valve and inlet pump
Water distance	Around or less than 3 cm	Turn OFF inlet valve and inlet pump
pH value	Greater than 6.5	Turn ON nutrient dosing pump for controlled duration
TDS value	Around 1500 ppm	Start outlet valve and outlet pump operation
Drain level	Approximately 6 cm to 7 cm	Turn OFF outlet valve and outlet pump

3.7 Web Dashboard Design Methodology

3.7.1 Separate ESP32 Reading Cards

The web dashboard interface design provided information about each ESP32 board individually. The reading of each individual ESP32 board included sensors readings, status of the board, time since it was last seen, Wi-Fi signal strength, status of the actuator, control mode, and status of logic control. It made information more readable and allowed the user to track which exact board sent out the particular reading.

The dashboard did not aggregate sensor readings. It was essential for testing and troubleshooting purposes as aggregation of values coming from different ESP32 boards could have concealed any failures or abnormal values. The interface of a single ESP32 reading card is illustrated in Figure 3.2 below.

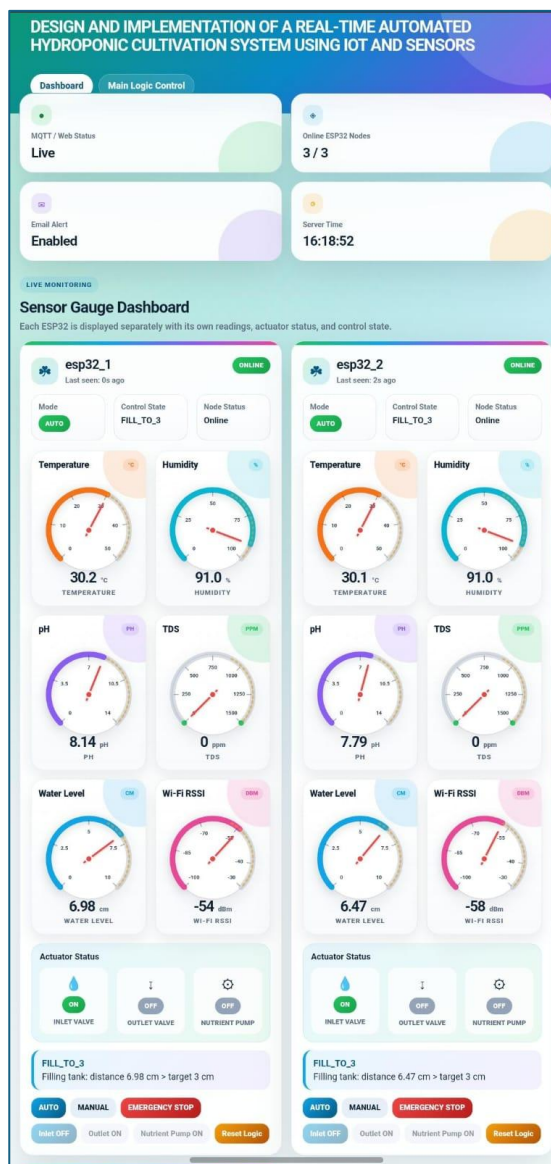


Figure 3.2 Web dashboard with separate ESP32 reading cards

3.7.2 Manual Control and Emergency Stop

The dashboard comprised manual controls for opening and closing the inlet valve, outlet valve, dosing pump, inlet pump, and outlet pump. Auto/manual switch and an emergency stop button also constituted the dashboard. Both these features were important in test and maintenance mode because the operator had control over the operations at their own will. The emergency stop button would ensure that all the actuators switched OFF so that no further automatic activity occurred.

3.8 Mobile Application Methodology

The application was embedded with the backend server, providing for remote control and monitoring capabilities using the mobile application. In connecting with the backend server on Raspberry Pi through the set remote access route, the user was able to observe the crucial hydroponics parameters and actuator status without being limited to the web dashboard alone.

The most recent iteration of the mobile application incorporated the use of the side panel with ESP32 1, ESP32 2, and ESP32 3 sections. These sections featured individual display of each ESP32 unit alone, along with the readings, controller state, actuator status, and control buttons. The problem previously faced by selecting ESP32 2 or ESP32 3 and getting ESP32 1 was addressed with this new design feature.

3.9 Network and Deployment Methodology

3.9.1 Static IP and Local Network Configuration

The Raspberry Pi had a static IP address to make sure that there was stability in the connection between the server and the ESP32 boards. In case the Raspberry Pi did not have a static IP address, any router restart would cause a change in its address and disrupt the MQTT connection.

3.9.2 Wireless VNC Access

VNC connection setup enabled remote control of the Raspberry Pi through the local network. It facilitated control of the Raspberry Pi desktop without any physical hardware like a screen, keyboard, or mouse being connected. It proved beneficial for software debugging and testing.

3.9.3 Docker-Based Deployment

Docker was used in deploying and managing the backend services. Docker provided an efficient way of keeping the backend environment organized through packaging the application and its required dependencies.

3.9.4 Cloudflare and Domain-Based Remote Access

The remote access was established by utilizing domain services and the Cloudflare tunneling technique. In this way, the locally hosted dashboard and backend system could be reached remotely. This setup helped to improve flexibility in monitoring and to demonstrate the project, without the need for relying solely on local network access. As a precautionary measure, the dashboard needs to be secured while it can be reached via a remote tunnel, while the MQTT broker needs to stay on the local network.

3.10 Testing and Validation Methodology

3.10.1 Sensor and MQTT Testing

In the initial test phase, it was confirmed whether all ESP32 were able to collect data from their respective sensors and publish it to the Raspberry Pi using MQTT protocol. The MQTT server was tested to check whether the JSON messages were received successfully by the Server.js server.

3.10.2 Relay and Actuator Testing

Secondly, relay operation and actuation of each component was done to confirm proper functioning. Individual components such as the inlet valve, outlet valve, fertilizer feed pump, inlet pump, and outlet pumps were checked individually. The valve and pump coordination was tested through actuation of the respective pumps when signals from the valves were sent to the ESP32.

3.10.3 Dashboard and Mobile Application Testing

Tests were conducted to validate that individual ESP32 cards operated independently. Tests included online/offline status of devices, last-seen timestamp, sensors readings, valve operation, pump operation, and logical status. Tests for manual operation, automatic/manual switch, resetting of logic, and emergency stop operation were also performed. Tests were conducted to validate communication via the mobile app using the backend server.

3.10.4 Full Automation Cycle Testing

The last test phase ensured that the entire hydroponics automation cycle was working fine. Testing was done on water level monitoring, inlet valve and inlet pump working, pH testing, fertilizer dosage pump control, TDS testing, outlet valve and outlet pump control, drainage, refill, dashboard update, and mobile app communication.

3.11 Summary of Design Methodology

The design process followed a hierarchical IoT design approach involving ESP32-based sensors/actuators nodes, MQTT communication, Raspberry Pi 5-based backend control, web-based dashboard monitoring, mobile application compatibility, and remote deployment. The important design consideration included the employment of individual ESP32-based reader modules, without averaging, that allowed for more accurate and effective monitoring and debugging of the nodes. Additionally, a new synchronization mechanism of the valve-pumps based on automatic triggering of the pumps whenever the corresponding valves' signals were received was implemented. As a result, the designed methodology allowed for building a fully-featured IoT-based system for real-time hydroponics cultivation.

4 IMPLEMENTATION

The chapter elaborates on the actual implementation of the real-time automated hydroponic cultivation system involving IoT and sensors. The implementation phase mainly aimed to translate the design concept to the operational model by deploying ESP32 sensor/control nodes, Raspberry Pi 5 local server, Mosquitto MQTT broker, server.js backend server, web dashboard, mobile interface, relay modules, solenoid valves, water pumps, and nutrient dosing pump.

Member 2's main task during the implementation process involved the implementation of the Raspberry Pi 5 local server, MQTT connectivity and messaging, Server.js backend server, dashboard, mobile interface, networking and deployment, and remote access features of the system. The system has been developed to receive real-time data from ESP32 nodes, process data in the Raspberry Pi server, display real-time data on the dashboard and execute automatic control of water levels, pH, TDS, valve operation, and nutrient dosing.

The actual implementation workflow of the hydroponics system is illustrated below as shown in Figure 4.1.

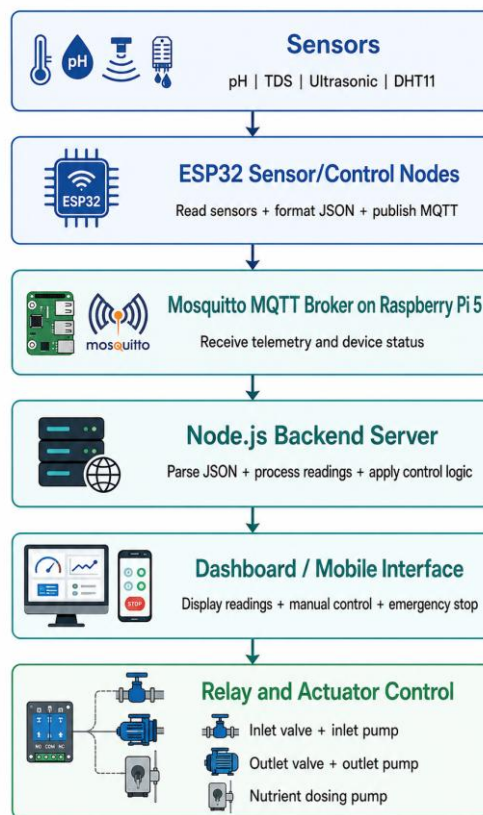


Figure 4.1 Overall implementation workflow of the hydroponic system

4.1 Implementation Environment

The execution environment had hardware parts, embedded systems, software elements, communication channels, and user interface components. The ESP32 development boards were used to perform sensor interfacing and control the actuators, while the Raspberry Pi 5 was used as the local server. The Server.js backend and Mosquitto MQTT broker were hosted on the Raspberry Pi to facilitate real-time communication and decision-making.

4.1.1 Hardware Implementation Environment

The hardware implementation included the ESP32 boards, Raspberry Pi 5, sensors, relay modules, solenoid valves, water pumps, and supporting power supply components. The hardware components used in the implementation are listed in Table 4.1.

Table 4.1 Hardware components used in the implementation

Hardware Component	Quantity / Description	Purpose
ESP32 boards	3	Sensor reading, MQTT publishing, and actuator control
Raspberry Pi 5	1	Local server and central controller
pH sensor	1 or more	Measuring nutrient solution pH
TDS sensor	1 or more	Measuring nutrient concentration
Ultrasonic sensor	1 or more	Measuring water level distance
DHT11 sensor	1 or more	Measuring temperature and humidity
Inlet solenoid valve	As required	Controlling water inlet flow
Outlet solenoid valve	As required	Controlling water outlet flow
Relay modules	As required	Switching valves and pumps
Nutrient dosing pump	1	Dosing nutrient solution based on pH condition
Inlet pump	1	Supporting water filling operation
Outlet pump	1	Supporting water draining operation
12 V DC power supply	1	Supplying power to pumps and valves
Jumper wires and circuit components	As required	Hardware connection and testing

4.1.2 Software Implementation Environment

The software environment was selected to support embedded data acquisition, MQTT communication, backend processing, dashboard display, mobile compatibility, deployment, and remote access. The main software tools and technologies used are shown in Table 4.2.

Table 4.2 Software tools and technologies used

Software / Technology	Purpose
Raspberry Pi OS	Operating system for Raspberry Pi 5
Arduino IDE / PlatformIO	ESP32 firmware development
Mosquitto MQTT Broker	MQTT communication between ESP32 and Raspberry Pi
Server.js	Backend server implementation
HTML, CSS, JavaScript	Web dashboard development
Mobile application interface	Mobile-based monitoring and control
Docker	Backend deployment and service management
VNC	Wireless Raspberry Pi desktop access
Static IP configuration	Stable local network communication
Cloudflare Tunnel	Remote access to local dashboard/backend
Domain service	User-friendly remote dashboard access

4.2 ESP32 Sensor Node Implementation

The ESP32 devices were designed using distributed sensor and controller nodes. The ESP32 devices captured the readings from the attached sensors and sent the values to the Raspberry Pi via the MQTT protocol. Also, ESP32 nodes were configured for the operation of relays for the control of solenoid valves, pumps, and actuators.

Figure 4.2 illustrates the design of the ESP32 sensor node. The figure depicts the communication path of sensors, ESP32, MQTT, and relay-operated actuators.

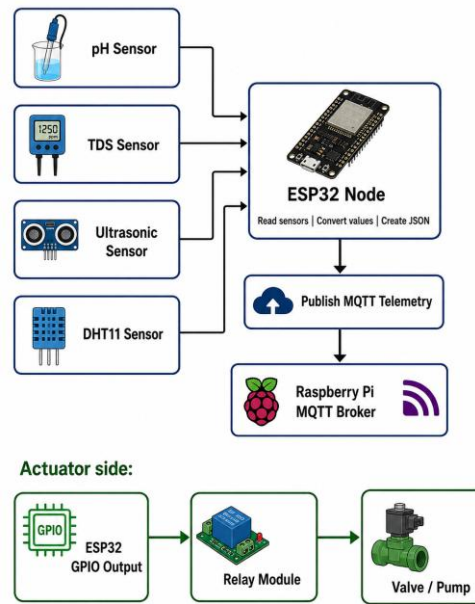


Figure 4.2 ESP32 sensor node implementation diagram

4.2.1 Sensor Interfacing

Connection of ESP32 boards was done to pH sensor, TDS sensor, ultrasonic water level sensor, and DHT11 temperature and humidity sensor. Since the pH and TDS sensors provide an analog output, they were connected to ESP32 board analog pins. On the other hand, the ultrasonic sensor was connected through different GPIO pins (trigger and echo pins) to calculate the water distance. Lastly, the DHT11 temperature and humidity sensor was connected to one GPIO pin. The ESP32 GPIO pins for interfacing sensors and actuators are presented in Table 4.3.

Table 4.3 ESP32 sensor and actuator pin configuration

Device / Sensor	ESP32 GPIO Pin	Function
Inlet solenoid valve relay	GPIO 27	Inlet valve control
Outlet solenoid valve relay	GPIO 26	Outlet valve control
Nutrient dosing pump relay	GPIO 25	Nutrient pump control
TDS sensor	GPIO 35	TDS measurement
pH sensor	GPIO 34	pH measurement
Ultrasonic sensor Trig	GPIO 5	Trigger signal
Ultrasonic sensor Echo	GPIO 18	Echo signal
DHT11 sensor	GPIO 4	Temperature and humidity measurement

4.2.2 JSON Data Formatting

The data from sensors was sent in JSON payloads before publishing it on the MQTT broker server. The reason why JSON format was used was that it was possible to send data from sensors as well as other data like the state of actuators, device name, and their statuses in the JSON format.

An example of a JSON payload that would be used for ESP32 is presented below:

```
{
  "device": "esp32_1",
  "temperature": 28.5,
  "humidity": 72,
  "ph": 6.8,
  "tds": 1450,
  "water_distance": 3.2,
  "inlet_status": "OFF",
  "outlet_status": "OFF",
  "wifi_rssi": -55
}
```

4.2.3 MQTT Data Publishing

Individual ESP32 devices sent their respective sensor values and actuator statuses to the MQTT broker running on the Raspberry Pi 5. Individual topics were defined for each ESP32 device for individual monitoring purposes to avoid mixing sensor values from different ESP32 devices.

Figure 4.3 below shows the process of data exchange between the ESP32 devices, Raspberry Pi, backend server, and dashboard using MQTT protocol.

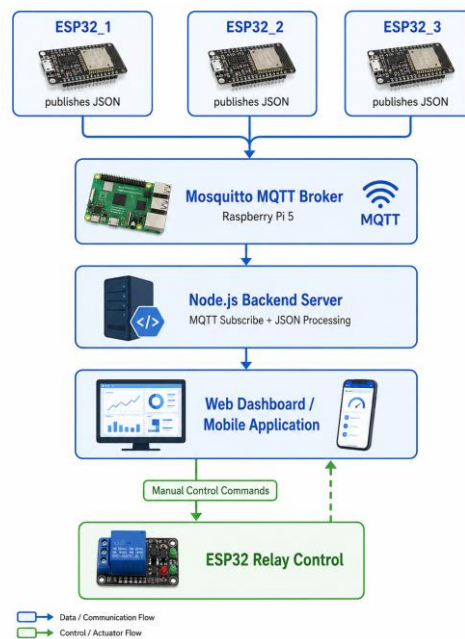


Figure 4.3 MQTT communication flow between ESP32, Raspberry Pi, backend, and dashboard

Table 4.4 MQTT topic structure and JSON payload description

MQTT Topic	Direction	Payload / Description
esp32/esp32_1/telemetry	ESP32 to Raspberry Pi	Sensor readings from ESP32 1
esp32/esp32_2/telemetry	ESP32 to Raspberry Pi	Sensor readings from ESP32 2
esp32/esp32_3/telemetry	ESP32 to Raspberry Pi	Sensor readings from ESP32 3
esp32/+/status	ESP32 to Raspberry Pi	Device online/offline status
esp32/+/logs	ESP32 to Raspberry Pi	Serial log or debugging messages
control/inlet	Raspberry Pi to ESP32	Inlet valve control command
control/outlet	Raspberry Pi to ESP32	Outlet valve control command
control/nutrient_pump	Raspberry Pi to ESP32	Nutrient dosing pump command
control/mode	Dashboard to backend / ESP32	Auto/manual mode selection
control/emergency_stop	Dashboard to backend / ESP32	Emergency stop command

4.3 Raspberry Pi 5 Local Server Implementation

Raspberry Pi 5 was used as the local server and overall controller for the hydroponics system. This device housed the Mosquitto MQTT broker and Server.js backend server. In addition, it served as the main connection for the ESP32s, web dashboard, mobile application, and other remote access services.

The architecture of the local server deployment is depicted in Figure 4.4 below.

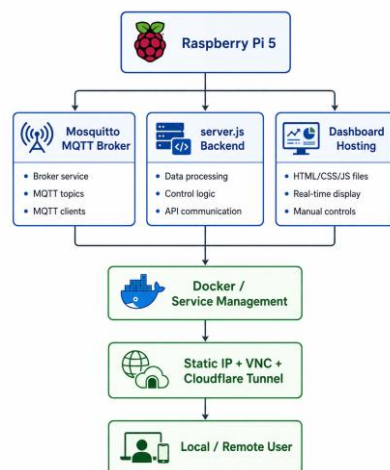


Figure 4.4 Raspberry Pi 5 local server deployment structure

4.3.1 Raspberry Pi Operating System Configuration

OS on the Raspberry Pi 5 was installed and configured. Preparation of the Raspberry Pi 5 was done in preparation for the running of a local server. Interfaces such as SSH, VNC, and network were enabled for the Raspberry Pi 5. Through SSH, one could have access to the Raspberry Pi via the command line while VNC provided wireless access to the desktop during testing.

Required packages were installed on the Raspberry Pi 5 before setting up the MQTT broker and dependencies on the same.

4.3.2 Mosquitto MQTT Broker Installation and Configuration

The Mosquitto MQTT broker was deployed on the Raspberry Pi system to facilitate MQTT messaging between ESP32 boards and the backend server. The MQTT broker was configured to receive MQTT data coming from ESP32 devices located within the same local network.

The ESP32s published their sensor data to the appropriate topics. The backend server subscribed to the respective topics to get access to the incoming sensor data.

4.3.3 Network Configuration

A static IP address was assigned to the Raspberry Pi so that the ESP32 modules will always have the same MQTT broker address. Without assigning the static IP address, the IP address of the Raspberry Pi may vary after router rebooting, leading to the inability to communicate with each other.

Moreover, wireless access to the Raspberry Pi desktop via VNC was set up so that control can be performed without using a screen, keyboard, and mouse. This became particularly convenient during the testing process when the system had to be accessed remotely from inside the local network.

4.4 Server.js Backend Server Implementation

Server.js backend server was developed to serve the connection between MQTT broker, dashboard, mobile app, and controlling code. It subscribed to MQTT channels, got JSON messages, performed sensor data processing, monitored ESP32 states, and sent updated data to UIs.

4.4.1 MQTT Subscription Handling

The backend server was configured to subscribe to telemetry, status, and logging events from the ESP32 nodes. On receiving a new MQTT message, the backend would extract the device ID, parse the message content, and then save the extracted values corresponding to the specific ESP32 device.

This design made it possible for each ESP32 chip in the dashboard to display its readings separately. In case any device stopped sending events, the backend server would use the last seen time stamp to deduce that the ESP32 was offline or waiting for telemetry.

4.4.2 Sensor Data Processing

These sensor data of pH, TDS, water distance, temperature, humidity, relay status, WiFi signal strength, and device status were processed by the backend. The latest sensor data were kept in memory on the backend and accessible to the dashboards and the mobile app.

The readings from sensors were compared with control parameters as well. For instance, water distance determined whether the inlet valves opened, the pH value controlled nutrient dosing, and TDS controlled whether the drain cycle should start.

4.4.3 API and Dashboard Communication

The backend served as the communication channel for Raspberry Pi, web dashboard, and mobile app. Real-time data was updated on the dashboard, while the backend received control commands from the user interface manually entered by the user.

Commands that could be issued from the dashboard included inlet valve ON/OFF, outlet valve ON/OFF, nutrient pump ON/OFF, auto/manual switch ON/OFF, reset logic, and emergency stop command.

4.5 Web Dashboard Implementation

The web dashboard was created utilizing HTML, CSS, and JavaScript languages. The purpose of the dashboard was to show sensor readings, status of actuators, ESP32 module status, state of the logic, and manually operated switches. Moreover, the dashboard was made mobile-friendly so that it could be monitored on a mobile device.

The layout of the dashboard and its mobile version can be seen in Figure 4.6 below. The layout was created in such a way that ESP32 readings were separated and controlled manually.

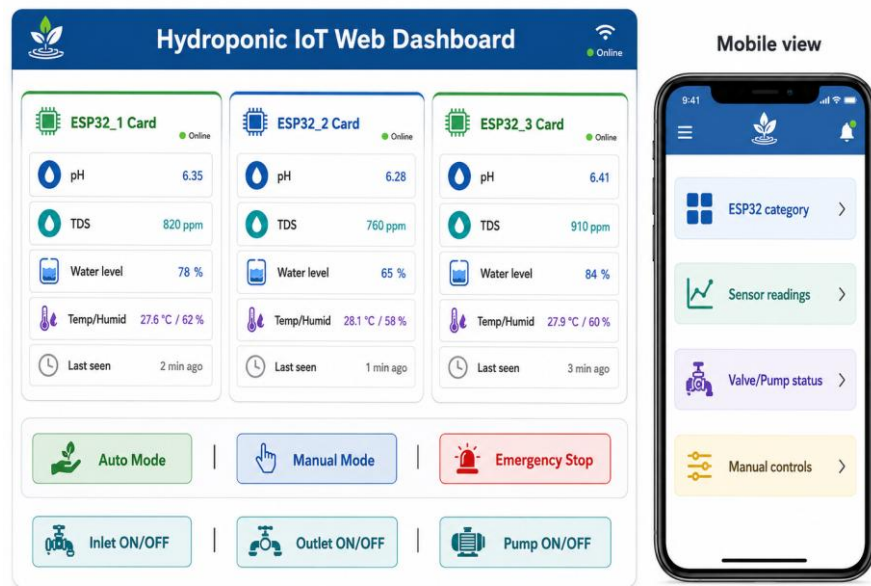


Figure 4.5 Web dashboard and mobile interface implementation layout

4.5.1 Real-Time Monitoring Interface

The live values of pH, TDS, water level, temperature, humidity, Wi-Fi RSSI, status of valves and pumps, as well as whether ESP32 is online or offline, appeared on the dashboard. They were obtained via Server.js backend with every incoming MQTT message.

4.5.2 Manual Control Interface

Manual controls in the form of buttons were incorporated for operating the system during tests, servicing, and emergencies. User controls involved manually switching ON and OFF the inlet valves, outlet valves, and the nutrient pump. The auto/manual control feature, as well as the emergency stop button, was incorporated into the dashboard.

User controls had priority over automatic controls during manual mode operation. Operation during emergency stop mode was such that no actuator output is ON.

4.5.3 ESP32 Category-Based Display

The dashboard was modified to monitor ESP32 in terms of categories. Individual ESP32s were presented on the dashboard as individual cards that showed their values and control status. In such a way, averaging was not needed, and the user could see whether an ESP32 is online, offline, or waiting for data.

The proposed solution facilitated the process of finding out errors in the sensors or communication channels as errors could be assigned to an ESP32 in particular.

4.5.4 Mobile Responsive Interface

The interface design of the dashboard was made responsive such that the screen would be accessible from mobile phones as well. The layout of the display was changed such that ESP32 cards and other controls would appear clearly from the mobile screens as well. This was to enhance ease of accessibility for the user without necessarily being tied to a desktop screen.

4.6 Actuator and Relay Control Implementation

The relay modules were responsible for controlling the inlet solenoid valve, outlet solenoid valve, nutrient dosing pump, inlet pump, and outlet pump. Control signals were generated by ESP32 GPIO pins that were sent to the relays, which turned on/off the actuators' circuits.

4.6.1 Inlet and Outlet Valve Control

Inlet solenoid valve was interfaced with a relay using GPIO pin 27 whereas the outlet solenoid valve was interfaced with a relay using GPIO pin 26. On triggering the inlet valve, the entry of water into the hydroponics system was facilitated whereas on activation of the outlet valve, water drainage took place from the system.

Furthermore, the new design enabled the operation of both inlet and outlet pumps along with the respective solenoid valves. If an active input of inlet valve was recognized by the ESP32 board, the inlet pump relay would be activated while similarly, detection of an outlet valve input would activate the outlet pump relay.

4.6.2 Nutrient Dosing Pump Control

The dosing of nutrients was achieved by the use of a relay that was wired to pin 25 on ESP32. The nutrient dose was performed based on the control criteria used by the Raspberry Pi server. If the pH level was greater than 6.5, then the server gave permission to perform the nutrient dose in a limited time interval.

It was necessary since the prolonged operation would have disrupted the nutrient balance. The mixing took place after the dosing procedure.

4.7 Automatic Control Logic Implementation

The automatic control system logic was used for controlling the water level, monitoring pH, testing TDS, and performing another draining and refilling cycle. The above process was done via the control system logic running on the Raspberry Pi backend that uses data provided by the ESP32 modules. The automatic control system flowchart is depicted in Figure 4.5 below.

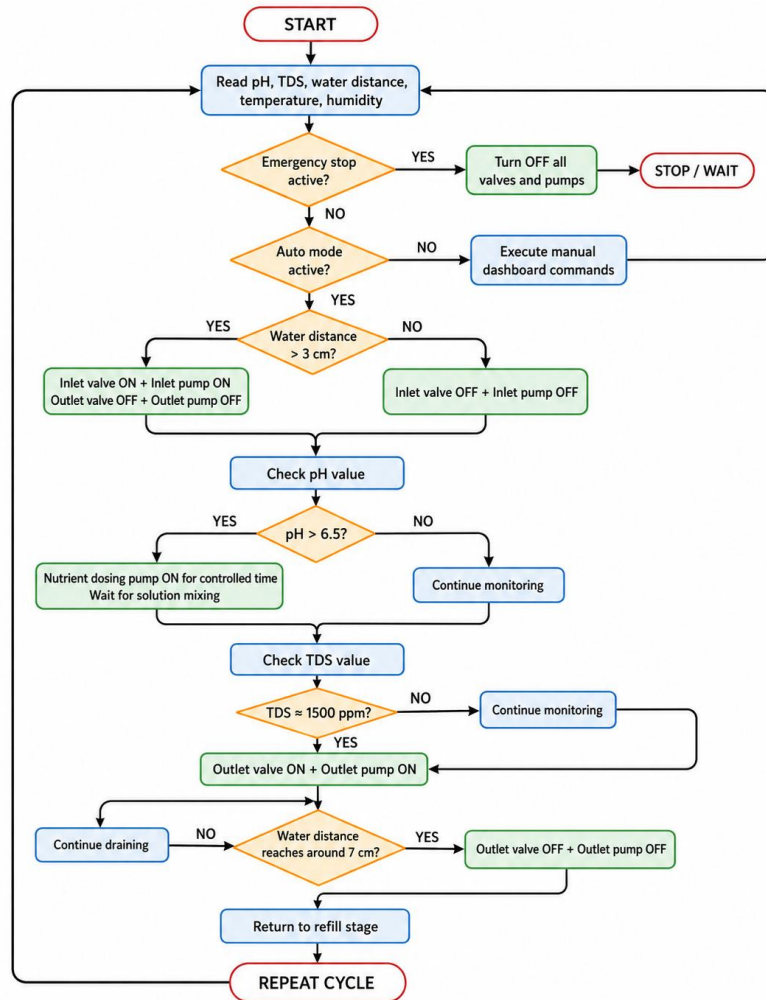


Figure 4.6 Automatic hydroponic control logic flowchart

4.7.1 Water Level Control

Water level control was the first step in this logic. The ultrasonic sensor was used to measure the distance from the sensor to the surface of the water. If the distance exceeded 3 cm, the system would recognize that the water level is low, in which case, the inlet valve and the inlet pump are set to ON.

When the water distance attained the target distance of around 3 cm, the inlet valve and inlet pump were turned OFF.

4.7.2 pH Monitoring and Alert Logic

Following the stabilization of the water level, the pH value was tested. The acceptable range of pH value was assumed to be around pH 7. In cases where the pH value was not within the acceptable range, then an alert condition was generated by the back-end system. If the pH value was more than 6.5, then the nutrient dosing pump was turned on based on the designed dose range.

The alert function proved helpful in notifying any problems of pH value or system issues.

4.7.3 TDS-Based Control

Once the pH levels stabilized, the TDS level was then measured. The ideal TDS level was estimated at around 1500 ppm. The system would wait for the TDS levels to stabilize at the desired range before proceeding with the outlet draining phase.

This was done to ensure that the nutrient levels were monitored prior to water circulation.

4.7.4 Drain and Refill Cycle

For the fulfillment of the TDS condition, both the outlet valve and outlet pump would be switched ON. The process of water discharge would continue until the ultrasonic distance became approximately 7 cm. Once the drain level had been attained, both the outlet valve and outlet pump were switched OFF.

Afterwards, again the inlet valve and inlet pump would operate up until when the water level becomes about 3 cm. This is how an entire automatic hydroponic operation cycle takes place; the whole process would repeat automatically.

Main control conditions applied for the automation system are shown in Table 4.5.

Table 4.5 Main control conditions used in the automation logic

Parameter / Condition	Threshold / Target	Control Action
Water distance low condition	Greater than 3 cm	Turn ON inlet valve and inlet pump
Required water level	Approximately 3 cm	Turn OFF inlet valve and inlet pump
pH target	Around pH 7	Continue monitoring
pH dosing condition	Greater than 6.5	Activate nutrient dosing pump for controlled time
TDS target	Approximately 1500 ppm	Start outlet drain process
Drain level	Around 7 cm	Turn OFF outlet valve and outlet pump
Emergency stop	Activated by user	Turn OFF all actuators
Manual mode	Selected by user	Execute dashboard/mobile control commands

4.8 Email Alert and Remote Monitoring Implementation

The use of email alerts as part of monitoring was implemented optionally where any abnormal pH levels or off-line status would trigger an alert from the backend system. This was important since it made the system safe to use as it would notify the user of any abnormalities within the system.

Remote monitoring was achieved through network settings and Cloudflare tunneling. For instance, network settings ensured that the server hosted by the Raspberry Pi could be accessed remotely using a domain name and Cloudflare tunneling to access the dashboard interface when off-campus.

4.9 System Deployment

Deployment was done in the Raspberry Pi 5. Docker was employed in structuring the backend and making it easy for deployment management purposes. The ease involved managing the backend service in terms of starting and restarting it where necessary.

The steps involved in deployment were: configuration of Raspberry Pi OS, installation of packages, Mosquitto MQTT broker setup, installation of Server.js dependencies, backend server execution, hosting of dashboard, static IP settings, VNC Access, and Cloudflare domain remote access.

4.10 Chapter Summary

Implementation of the Automated Hydroponic Cultivation System is detailed in this chapter. This includes the configuration of the ESP32 sensor nodes, formatting of the JSON payload, sending the data via MQTT, setup of Raspberry Pi 5 local server, configuration of the MQTT broker (Mosquitto), Server.js backend, integration with the web dashboard, responsive mobile interface, controlling the actuators using relays, automatic control of the water level, monitoring and controlling pH, TDS control, nutrient dosing, alert handling, remote monitoring, and deployment using Docker.

The tasks executed by Member 2 during implementation mainly comprised the Raspberry Pi 5 server, MQTT messaging, Server.js backend, dashboard integration, mobile interface, networking, deployment, remote access, and executing the logic. This implementation integrated the ESP32 nodes, sensors, Raspberry Pi backend, dashboard, mobile application interface, relays, valve controllers, pump controls, and nutrient dosing pump to implement a real-time system.

5 INTEGRATION AND TESTING

In this chapter, the integration and testing of the real-time automated hydroponics growing system that utilizes IoT technology and sensors is discussed. This phase aimed at testing whether the ESP32 sensor/control modules, the Raspberry Pi 5 local server, the Mosquitto MQTT broker, the Server.js backend server, the dashboard, mobile interface, the sensors, relays, solenoid valves, and other remote monitoring capabilities worked as a cohesive system.

The core task of Member 2 in this phase involved integration and testing of the Raspberry Pi 5 local server, MQTT communication, Server.js backend server, dashboard communication, mobile interface, ESP32 communications, local networking, remote monitoring, and system level testing. The process of testing was conducted incrementally starting from single module testing towards complete end-to-end system testing.

5.1 System Integration Overview

The system was integrated using a hierarchical approach. Data from the sensors concerning the pH levels, total dissolved solids, ultrasonic water levels, temperature, and humidity levels were acquired. These values were analyzed by ESP32 nodes, which sent the results via MQTT messaging in JSON format to Raspberry Pi 5. The Raspberry Pi 5 was where the Mosquitto MQTT broker and Server.js backend server ran. The backend server processed data from the sensors and controlled the dashboard and mobile application interface.

Figure 5.1 below displays the arrangement of the entire system integration process.

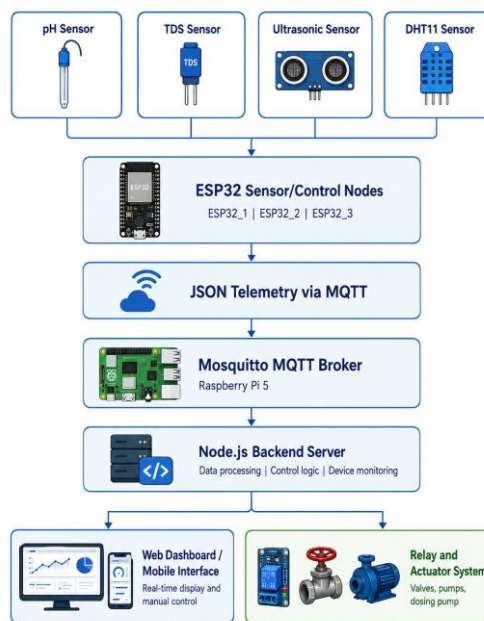


Figure 5.1 Overall system integration diagram

The main integrated components and their functions are summarized in Table 5.1. This table explains the role of each component in the complete system.

Table 5.1 Integrated system components and their functions

Integrated Component	Function in the System	Integration Purpose
ESP32 sensor/control nodes	Read sensors and control relays	Sensor acquisition and actuator control
Raspberry Pi 5	Local server and main controller	Central processing and service hosting
Mosquitto MQTT broker	Handles MQTT publish/subscribe messages	Communication between ESP32 nodes and backend
Server.js backend server	Processes sensor data and executes control logic	Backend control and dashboard communication
Web dashboard	Displays readings and control buttons	Real-time monitoring and manual control
Mobile-compatible interface	Provides mobile access to the dashboard	Portable monitoring and control
pH sensor	Measures nutrient solution pH	pH monitoring and dosing decision
TDS sensor	Measures nutrient concentration	TDS-based control decision
Ultrasonic sensor	Measures water distance	Water level regulation
DHT11 sensor	Measures temperature and humidity	Environmental monitoring
Relay modules	Switch actuator circuits	Control valves and pumps
Inlet valve and inlet pump	Support filling operation	Water level control
Outlet valve and outlet pump	Support draining operation	Drain cycle control
Nutrient dosing pump	Adds nutrient solution	pH-based dosing operation
Local Wi-Fi network	Connects ESP32 nodes and Raspberry Pi	Local communication
Remote access configuration	Enables external dashboard access	Remote monitoring and demonstration

5.2 Hardware Integration

In this hardware integration phase, ESP32 boards, sensors, relay modules, solenoid valves, inlet pump, outlet pump, and nutrient dosing pumps were interfaced with each other. The connections for pH sensors, and TDS sensors were done to an analog input pin of ESP32 board. For ultrasonic sensor, trigger and echo pin were used to measure distance of the water.

To control the operation of inlet solenoid valve, outlet solenoid valve, nutrient dosing pump, inlet pump, and outlet pump, relay modules were used which could switch high power actuators using low power ESP32 control signals. Wiring of the system was checked first before checking its operation.

Figure 5.3 illustrates the hardware and software integration phase workflow. This diagram depicts the order of operations while connecting the hardware with software part.

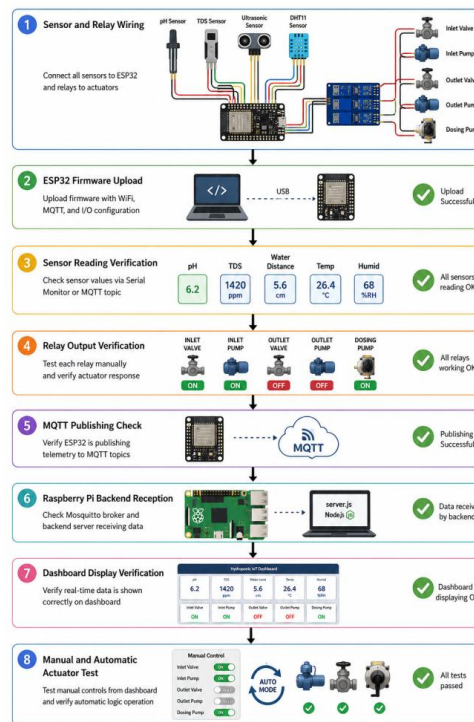


Figure 5.3 Hardware and software integration workflow

The hardware integration test demonstrated that the sensor readings could be collected through the ESP32 and that the relays could be turned on/off accurately. Following this test phase, the actuators were finally attached to the system, and manual control was initially performed.

5.3 Software and Communication Integration

Software and communication integration included the following components: Raspberry Pi 5 board setup, Mosquitto MQTT broker configuration, server.js backend server, MQTT topic management, JSON payload processing, dashboard communication, and integration of a mobile-friendly user interface. The ESP32 nodes transmitted the telemetry data as JSON payloads to the MQTT broker, while the Server.js backend server subscribed to necessary MQTT topics and processed incoming data.

The flow of communication using the MQTT protocol is illustrated in Figure 5.2. This procedure helped validate whether the ESP32 nodes, MQTT broker, backend server, and dashboard were communicating appropriately.

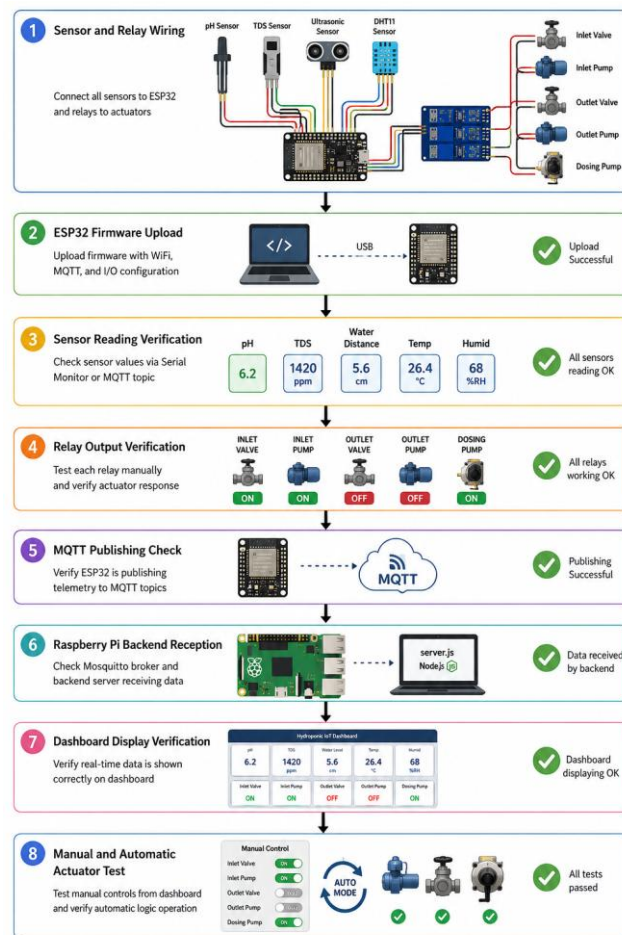


Figure 5.3 MQTT data communication testing sequence

MQTT communication and data validation test cases are presented in Table 5.2. Such tests were conducted to make sure that all ESP32 nodes properly transfer data and also that this data is allocated to the appropriate dashboard card.

Table 5.2 MQTT communication and data verification test cases

Test Case	Test Objective	Test Procedure	Expected Result	Actual Result	Status
ESP32 Wi-Fi connection	Verify local network connection	ESP32 was powered and connected to the router	ESP32 connects to Wi-Fi	ESP32 connected successfully	Pass
MQTT broker connection	Verify broker access	ESP32 was configured with Raspberry Pi broker IP	ESP32 connects to Mosquitto broker	Broker connection established	Pass
ESP32_1 telemetry	Verify ESP32_1 topic	ESP32_1 published JSON data	Data received under ESP32_1	Data received correctly	Pass
ESP32_2 telemetry	Verify ESP32_2 topic	ESP32_2 published JSON data	Data received under ESP32_2	Data received correctly	Pass
ESP32_3 telemetry	Verify ESP32_3 topic	ESP32_3 published JSON data	Data received under ESP32_3	Data received correctly	Pass
JSON payload format	Verify readable data structure	Backend parsed received payload	JSON values separated correctly	Payload processed correctly	Pass
Device status update	Verify online/offline status	MQTT publish interval was monitored	Last-seen time updates	Device status updated	Pass

Results demonstrated that communication through MQTT occurred successfully between the ESP32 nodes and the Raspberry Pi server. Independent readings on the ESP32 were achieved by implementing different topics and unique IDs for each device.

5.4 Dashboard and Mobile Interface Testing

Testing for the web dashboard and mobile-friendly interface was done after ensuring that the backend had successfully received MQTT data. The requirements for testing the dashboard included displaying parameters like pH value, TDS, water distance, temperature, humidity, Wi-Fi RSSI, relay state, actuator state, ESP32 status, and control mode. The other requirement was for the dashboard to offer manual control functions and emergency stop.

The flow diagram for testing manual control from dashboard to actuator is illustrated in Figure 5.4.

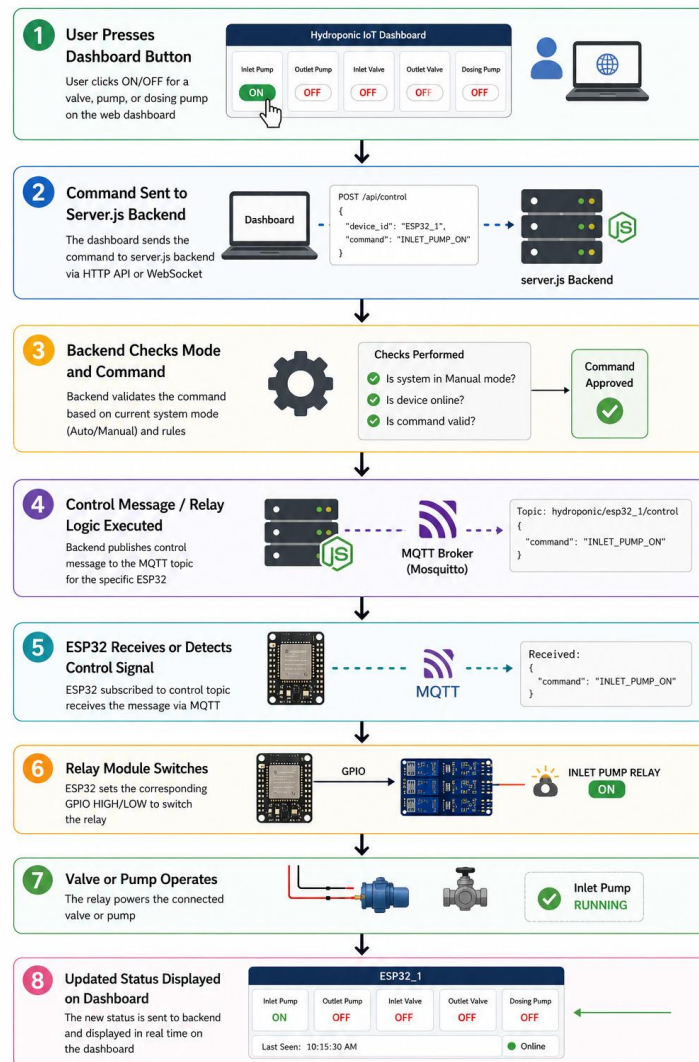


Figure 5.4 Dashboard-to-actuator manual control test flow

The dashboard and mobile interface test cases are summarized in Table 5.3. These tests were used to verify real-time display, manual control, ESP32 category-wise monitoring, and mobile view compatibility.

Table 5.3 Dashboard and mobile interface test cases

Test Case	Test Objective	Test Procedure	Expected Result	Actual Result	Status
Real-time pH display	Verify pH display	ESP32 published pH reading	pH value updates on dashboard	pH displayed correctly	Pass
TDS display	Verify nutrient value display	ESP32 published TDS reading	TDS value updates	TDS displayed correctly	Pass
Water distance display	Verify ultrasonic reading	Water distance was published	Distance value updates	Distance displayed correctly	Pass
ESP32 card separation	Verify node-wise display	Each ESP32 published separately	Each card updates independently	Separate cards worked	Pass
Manual inlet control	Verify inlet button	Inlet ON/OFF button was pressed	Inlet actuator responds	Inlet command responded	Pass
Manual outlet control	Verify outlet button	Outlet ON/OFF button was pressed	Outlet actuator responds	Outlet command responded	Pass
Emergency stop button	Verify safety stop	Emergency stop was activated	All actuators stop	Actuators stopped	Pass
Mobile view	Verify mobile compatibility	Dashboard opened on mobile screen	Layout remains readable	Mobile view displayed correctly	Pass

The dashboard testing confirmed that the interface supported real-time monitoring and manual control. The separate ESP32 reading cards improved fault identification because each ESP32 node could be observed independently.

5.5 Actuator and Relay Testing

Tests for actuators and relays were conducted to determine whether each actuator functioned properly in controlling the inlet valve, outlet valve, inlet pump, outlet pump, and nutrient dosing pump. These tests were done separately from the automated control sequence test.

The inlet valve and inlet pump were tested by switching on the inlet control signal. It was expected that the inlet valve should open and the inlet pump should operate simultaneously. It was observed that the inlet pump switched on upon activation of the inlet valve control signal. The same process was repeated in determining the functionality of the outlet valve and outlet pump. Testing of the nutrient dosing pump involved its operation according to the pH dosing condition.

Table 5.4 Test results for actuators and relays

Actuator Test	Test Objective	Test Procedure	Expected Result	Actual Result	Status
Inlet valve ON	Verify inlet relay switching	Inlet command was sent ON	Inlet valve opens	Inlet valve operated	Pass
Inlet pump trigger	Verify inlet valve-pump link	Inlet valve signal was activated	Inlet pump turns ON	Inlet pump activated	Pass
Inlet OFF	Verify stop condition	Inlet command was sent OFF	Inlet valve and pump stop	Inlet system stopped	Pass
Outlet valve ON	Verify outlet relay switching	Outlet command was sent ON	Outlet valve opens	Outlet valve operated	Pass
Outlet pump trigger	Verify outlet valve-pump link	Outlet valve signal was activated	Outlet pump turns ON	Outlet pump activated	Pass
Outlet OFF	Verify stop condition	Outlet command was sent OFF	Outlet valve and pump stop	Outlet system stopped	Pass

Nutrient pump ON	Verify dosing relay	pH > 6.5 condition was applied	Nutrient pump activates	Pump activated for controlled time	Pass
Emergency stop	Verify safety shutdown	Emergency stop was pressed	All actuators turn OFF	All actuators stopped	Pass

The actuator test results confirmed that the relay modules responded correctly to control commands. The valve-pump coordination was also verified, which improved the reliability of filling and draining operations.

5.6 Automatic Control Logic Testing

Test of Automatic Control Logic was done in order to check the entire decision-making process within the system. This test was done for various activities like water level control, PH monitoring, nutrient dosing pump control, TDS monitoring, drain and refill process, manual override, and E-stop function.

5.7 Network and Remote Monitoring Testing

The network testing procedure was used to confirm that all the devices – ESP32 boards, Raspberry Pi 5, dashboard client, and the mobile version of the interface – could communicate successfully. The Raspberry Pi was assigned a static IP address, ensuring that the ESP32 nodes could always be publishing MQTT data to the right broker address.

First, the local Wi-Fi connection was tested. The expectation from the testing process was to check whether the ESP32 boards were communicating with the Raspberry Pi on the same network while constantly sending MQTT data to it. The outcome of the tests confirmed the local connectivity of devices.

Next, the delay of MQTT messages was measured. The delay between the publishing of MQTT messages and the update of the dashboard was insignificant because the dashboard refreshed almost immediately after the backend received MQTT messages.

Finally, remote access capabilities were tested using the previously established configuration. Remote access made it possible to use the dashboard even without being physically near the Raspberry Pi screen.

5.8 System-Level Testing and Results

The end-to-end system testing procedure was conducted after integration of all the hardware, software, communication modules, and actuators. The intention was to test the entire procedure from sensing to the control of the actuators, through monitoring of the process on the dashboard.

From the system-level tests, it was established that data from sensors could be sent by the ESP32 nodes, the Raspberry Pi backend could handle the received information accurately, and the dashboard displayed live readings accordingly. Tests on actuators verified that the inlet valve, outlet valve, inlet pump, outlet pump, and nutrient dosing pump were reacting to control mechanisms.

Overall performance of the setup was satisfactory for an implementation in a hydroponics setup. Use of an ESP32 display allowed for easier troubleshooting as individual nodes could be tested separately. Certain limitations were experienced during testing, which included communication delays when there were network issues and required calibration of some sensors. Nonetheless, despite these limitations, the setup was capable of performing its core functions.

5.9 Problems Encountered and Corrective Actions

Several practical issues were identified during integration and testing. The main problems and corrective actions are summarized in Table 5.6.

Problem Encountered	Possible Cause	Corrective Action Taken	Result
ESP32 failed to connect to MQTT broker	Incorrect broker IP or Wi-Fi issue	Static IP and MQTT broker address were checked	MQTT connection restored
Dashboard showed old values	Backend did not receive new telemetry	Last-seen and device status monitoring were added	Offline/waiting status improved
Sensor values appeared under wrong ESP32 card	Topic or device ID mismatch	MQTT topic and JSON device ID were verified	Correct card update achieved

Relay did not switch correctly	Wiring or GPIO configuration issue	Relay wiring and GPIO output were checked	Relay switching corrected
Pump did not run with valve signal	Signal scanning connection issue	Valve signal pin and pump relay trigger were tested	Pump coordination improved
Remote access failed	Tunnel or domain routing issue	Cloudflare tunnel and domain routing were checked	Remote access restored
Dashboard was difficult to view on mobile	Layout responsiveness issue	CSS layout was adjusted for mobile display	Mobile view improved
Emergency stop response needed verification	Command routing delay	Emergency stop command path was tested repeatedly	All actuator outputs stopped correctly

The corrective actions improved the stability of MQTT communication, dashboard display accuracy, actuator response, remote access, and safety operation. The testing process also helped confirm that the system was suitable for demonstration and further improvement.

5.10 Chapter Summary

The integration process of the proposed system was discussed in this chapter and included interconnection of ESP32 sensor/controller units, Raspberry Pi 5 computer, Mosquitto MQTT messaging broker, Server.js back-end server, web dashboard interface, mobile-compatible version of the interface, various sensors, relays, solenoid valves, pumps, nutrient dosing pump, local area network, and remote monitoring configuration.

The results of the testing proved the capability of ESP32 sensors to publish JSON messages using the MQTT protocol. Besides, the back-end server installed on the Raspberry Pi was tested in terms of receiving messages from the ESP32 nodes and performing calculations. Moreover, the web dashboard interface was able to display the real-time values received from sensors via ESP32 controller units separately. Relay and actuators tests revealed the proper functioning of the inlet valve, outlet valve, inlet pump, outlet pump, and nutrient dosing pump. Automated control logic testing proved the proper functionality of water level control, pH-based dosing, TDS, drain/refill, manual override, and emergency stop mechanisms.

6 RESULTS AND DISCUSSION

6.1 Introduction

This chapter will discuss the results and discussion regarding the real-time automated hydroponics growing system through IoT and sensor technology. The developed system is tested based on the performance of sensors, MQTT, Raspberry Pi 5 local server, web-based output dashboard, mobile-friendly user interface, actuator control, automatic controller algorithm, remote monitoring capability, and the overall performance of the whole system.

The primary objective of the testing phase was to see if the developed system could function as an integrated hydroponic growing system. The results discussed in this chapter focus mostly on the contribution of Member 2 that consists of Raspberry Pi 5 local server, Mosquitto MQTT service, Server.js backend server, dashboard incorporation, mobile-friendly user interface, networking, remote monitoring, data communication, and system testability.

6.2 Sensor Reading Results

From the sensor reading results, it is evident that ESP32 nodes managed to collect and relay the necessary parameters from the hydroponic systems. The inputs were obtained using pH sensor, TDS sensor, ultrasonic water level sensor, and DHT11 sensors. The sensor inputs were then relayed to the Raspberry Pi backend via MQTT protocol to be displayed on the dashboard.

The pH sensor reading is essential in monitoring the state of the nutrient solution. The hydroponic system was programmed to use the pH value to check if the nutrient dosing pump should be activated. According to the conditions used, the dosing was only started if the pH value exceeded 6.5. This proves that the pH sensor readings can be used to trigger nutrient dosing.

The TDS sensor reading is essential in monitoring the nutrient levels of the solution. In the hydroponic system, the TDS value is continuously monitored until it reaches around 1500 ppm. After reaching the required condition, the process of opening the outlet valves and activating the pumps can be started.

The ultrasonic sensor returned the water distance, which was among the most critical data in the automation procedure. Whenever the water distance was more than 3 cm, the automation system would determine that the water level was low and hence open the inlet valve and inlet

pump. Whenever the water distance approached 3 cm, the automation process would stop. In the draining process, the automation would monitor the water distance until the value was around 7 cm and then cease the process.

The temperature and humidity values appeared in the dashboard as per the readings from the DHT11 sensor. The readings served mostly as environmental parameters since they did not control any of the valves or pumps in the automation logic.

Generally, the sensor reading results were acceptable for automation use. However, the precision of pH and TDS readings relies highly on their calibration.

6.3 MQTT Communication Results

Results obtained from the MQTT communication were in agreement with assumptions regarding sending data from ESP32 sensors to the Raspberry Pi 5 using the Mosquitto MQTT broker. Indeed, each ESP32 board published JSON-payloads to the required topics that were being processed by the backend through subscribing.

Data structure defined within JSON payloads was easily readable. Every payload included information about the ID of the device, pH, TDS, water distance, temperature, humidity, Wi-Fi signal strength, and status of actuators. Therefore, the backend was capable of processing JSON-payloads and updating the values in the corresponding dashboard card.

Using MQTT topics made separation of the values for ESP32_1, ESP32_2, and ESP32_3 possible. It is worth mentioning that each of the devices should function independently. In other words, an average value was not calculated between different ESP32 boards. Therefore, a failure, data delay, or telemetry malfunction can be detected separately for each of them.

MQTT was suitable for near-real time monitoring. In fact, after receiving and processing the data through the MQTT connection, the dashboard was updated. Delays might be caused by Wi-Fi signal strength, routers, or ESP32 reconnection. However, these delays did not affect the functioning of the system since they were minor.

6.4 Web Dashboard and Mobile Interface Results

Real-time sensor values, status of devices, status of actuators, status of control, and buttons for manual control were all successfully shown on the dashboard. New ESP32 reading cards were added to update the dashboard so that each ESP32 would show its own readings separately from others. It enhanced the monitoring system as each ESP32 could now be checked by itself.

Important values such as pH, TDS, water distance, temperature, humidity, Wi-Fi RSSI, last-seen status, valve status, pump status, and logic state were available on the dashboard. Last-seen status was essential as it helped determine whether each ESP32 was transmitting data actively or waiting for telemetry. This feature could help find the reasons why some devices were not online while testing.

Manual control was used to open or close the inlet valve, open or close the outlet valve, start or stop nutrient dosing pump, switch between auto and manual modes, and stop operation by pressing the emergency stop button. Such control was needed to check actuators before the automatic cycle could be started.

Being mobile-compliant enabled the dashboard to be accessed via a mobile device. It helped increase usability as one was able to access the dashboard without having to rely on the use of a desktop or laptop computer alone.

In conclusion, the outcome from the development of the dashboard and its mobile component proved to be positive. The implementation of the separate ESP32 card enabled enhanced monitoring and troubleshooting capability.

6.5 Actuator Control Results

The actuator control tests proved that the inlet solenoid valve, outlet solenoid valve, inlet pump, outlet pump, and nutrient dosing pump could be run using relay modules. The relay modules would receive commands from the ESP32 and enable switching of the high power actuator circuits.

The inlet valve was opened during the filling operation mode. Whenever the sensor detected that there was low water level in the water tank, the inlet valve was turned ON. In the redesign, the inlet pump would operate automatically whenever the inlet valve command was ON. This

helped to improve filling efficiency as the water flowed with the help of the pump and not just gravity alone.

The outlet valve functioned in the drain operational mode. If the TDS was found to be above the set limit, then the outlet valve was turned ON along with the outlet pump. This was because the outlet pump would run whenever the outlet valve command was active.

Whenever the pH level was greater than 6.5, the nutrient dosing pump would be turned ON. The pump would be operated for a specified time period to prevent over dosing of nutrients in the solution.

The emergency stop function successfully turned OFF the valves and pumps during testing. This confirmed that the system had a safety function to stop actuator operation when required.

6.6 Automatic Control Logic Results

From the automatic control logic output results, the Raspberry Pi backend successfully executed operations on the received sensor data and made necessary decisions according to them. The control procedure involved water-level control, pH value measurement, dosing of nutrients, measuring the TDS value, drainage, refilling of nutrient solution, manual control, and emergency shutdown.

The control based on the water level functioned as expected according to the ultrasonic sensor. If the water distance was higher than 3 cm, then the inlet valve and inlet pump were turned ON. Once the necessary amount of water reached the tank, the inlet valve and inlet pump were turned OFF.

The pH value control operation also successfully functioned according to the conditions set for the dosing pump. In case the pH value was higher than 6.5, the dosing pump would be turned ON, and it would run for a certain period. Afterward, some time was given to ensure mixing of the nutrients before further checking.

The logic for monitoring TDS ensured that the TDS level became 1500 ppm before continuing. Once the TDS was at the required level, the outlet valve and outlet pump would switch ON. The drainage process would continue until the ultrasonic sensor became 7 cm away. At this point, the outlet valve and outlet pump would be turned OFF, and the process would begin once more.

Manual override meant that the dashboard could control the actuators when in manual mode. This helped during troubleshooting and maintenance operations. Emergency stop meant that the actuators would shut down, and no operation would be possible until the system had been reset. Thus, automatic control had successfully accomplished the desired operation of the hydroponic system.

6.7 System-Level Performance Discussion

In relation to system-level performance, the findings indicated that the integration of the ESP32 nodes, the Raspberry Pi 5 server, the Mosquitto MQTT broker, the Server.js backend, the dashboard, the mobile interface, the relay modules, the solenoid valves, the pumps, and the nutrient dosing pump into an integrated whole was possible. The sensor readings were taken from the ESP32 nodes, transferred via MQTT, processed at the backend level, presented on the dashboard, and used for making decisions.

The reaction time of the system was deemed satisfactory in relation to small-scale hydroponics. The MQTT protocol allowed nearly real-time data transmission, while the dashboard updates were triggered by the incoming messages from the MQTT broker. Furthermore, the ESP32 reading card helped better detect the problems associated with individual nodes. This finding was important due to the possibility of hiding the abnormalities of the node in case of its incorporation or averaging with other cards into the dashboard. The synchronization of the solenoid valves and pumps ensured better water movement.

Remote monitoring also enhanced the ease of use of the system since the dashboard could be accessed over the configured network. This enabled one to observe system statuses without interacting with the Raspberry Pi.

There was also some latency and variability noted during testing. These were primarily a result of delays in Wi-Fi communications, stabilization time of the sensors, mixing time of the water, and relay reaction time. These are common issues that may arise in practical IoT projects and embedded systems. Nevertheless, this did not affect the ability of the system to perform its intended functions.

6.8 Comparison with Expected Results

Comparisons were made between how the system should work and the actual results after testing the system. It was expected that the ESP32 cards would read sensor values and send those values to the Raspberry Pi using MQTT communication. This was accomplished since JSON sensor data were sent to the backend on the Raspberry Pi.

It was expected that the Raspberry Pi 5 would act as a server and the controller within the system. It was successful since a Mosquitto MQTT broker was run together with the Server.js server within the Raspberry Pi.

It was expected that the dashboard would show real-time sensor values, actuator states, device states, and manual controls. This was also achieved since each ESP32 node was shown independently using the separate ESP32 card setup, without averaging the values.

It was expected that the automatic control logic of the system will control the water level, pH based nutrient dosage, TDS level, drain function, refill, manual operation, and emergency stop. Testing showed that the system control logic is working according to design specifications. In conclusion, it can be noted that the system met its main goals.

6.9 Limitations Observed

Despite the success of achieving the project goals, there were some practical issues that came up. In particular, calibration was essential to the correct working of the pH and TDS sensors. Otherwise, the measurements could be incorrect and fail to indicate the actual state of the nutrient solution.

Another issue was associated with the need to ensure proper communication using Wi-Fi. The weak Wi-Fi signal or router disconnection may lead to MQTT connection problems. Although static IP addresses improved the communication performance, stable Wi-Fi coverage is still crucial.

Switching relays involved a small delay, which was tolerable within the limits of the hydroponic task at hand. However, more advanced relay switching options could be used in subsequent designs. Stable power supply of 12 V was also necessary to maintain optimal performance of all actuators.

Another limitation related to water mixing delay after the dosing of nutrients. Thus, some waiting time was needed before re-measurement of the sensor values.

Remote access relied on the presence of an internet connection and proper Cloudflare/domain setup. Without an internet connection, remote access would not be possible. Local dashboard access, however, could serve as an alternative.

The reading of the ultrasonic sensor could be inconsistent due to movements on the water surface. To solve this issue, one can filter the data or obtain several readings from the ESP32 before posting the result.

6.10 Summary of Results

The outcome revealed that the proposed IoT-based automated hydroponic cultivation system was able to meet the expected monitoring and control goals. The ESP32 nodes were successful in receiving sensor inputs and transmitting them as JSON through MQTT. Meanwhile, the Raspberry Pi 5 local server, MQTT broker, and Server.js backend were successful in receiving and processing the data sent to them.

There was effective display of the ESP32 inputs, actuator statuses, device status, and control functions on the web dashboard and mobile-friendly interface. The actuator output indicated the operation of the inlet valve, outlet valve, inlet pump, outlet pump, and nutrient dosing pump through relay control.

The automatic control logic successfully achieved water level control, pH-based nutrient dosing, TDS monitoring, draining and refilling, manual control, and emergency stop. In

conclusion, the designed system was effective and suitable as an automated monitoring and control platform for chilli and tomato cultivation.

Member 2 contribution was proven by the successful operation of the Raspberry Pi 5 server, MQTT, Server.js backend, dashboard implementation, mobile monitoring, networking, remote connection, and testing of the whole system.

7 CONCLUSION & DISCUSSION

7.1 Introduction

The concluding part of this chapter offers the summary of the findings of the project entitled "Design and Implementation of Real-time Automatic Hydroponic Cultivation System Using IoT & Sensors." It highlights the project's total outcome, including its success in accomplishing the set objectives. In addition, this section provides a brief overview of the limitations identified and the recommended improvements for the future.

7.2 Overall Project Conclusion

In conclusion, the implementation proved successful as the designed IoT-based automated hydroponics plant growth process has been accomplished. In particular, ESP32 sensor/controller modules, Raspberry Pi 5, Mosquitto MQTT, Server.js backend, web dashboard, mobile interface, various sensors, relay modules, solenoids, inlet and outlet valves, inlet and outlet pumps, and nutrient dosing pump have been included into the system design.

In addition, this system has succeeded in tracking different parameters essential for hydroponics, including water level, pH value, TDS, temperature, and humidity. ESP32 boards separately collected sensor values and transferred them via MQTT communication protocol in JSON form to the local server Raspberry Pi 5. The Raspberry Pi 5 worked as the local server and controller module, while Server.js processed data, updated the web dashboard, and made decision about the actions to be performed.

Besides, the implemented system successfully managed the inlet valve, outlet valve, inlet pump, outlet pump, and nutrient dosing pump automatically. Water level management, nutrient dosing based on the pH value, TDS measurement, drain and refill cycle, manual operation, and emergency stop have been realized as components of the automatic operation system. Thus, the purpose of the developed system has been fulfilled.

7.3 Achievement of Project Objectives

The first aim was to implement the development of a monitoring tool for hydroponic cultivation. This was accomplished by incorporating pH, TDS, ultrasonic water level, temperature, and humidity sensors into the ESP32 microcontrollers. These values were sent to the Raspberry Pi backend and displayed in the web-based and mobile application dashboards. The second aim was to develop Internet of Things communication between individual sensor nodes and the main controller. This was implemented by deploying an MQTT protocol broker Mosquitto on Raspberry Pi 5. ESP32 devices were used to send data as JSON objects to the server, and then this information was consumed on the Server.js backend side.

The third objective was the creation of automatic control tools for hydroponic water management and nutrition process control. This was implemented with the use of control logic incorporated in the backend that was responsible for managing water levels, triggering pH-dependent doses, monitoring TDS, and starting and stopping drain cycles. If the value of the pH meter was higher than 6.5, the nutrient dosing device was triggered. The drain cycle was started at approximately 1500 ppm of TDS.

The fourth goal was dashboard-based monitoring and control. It was attained by the use of a web dashboard which showed separate reading cards for the ESP32 sensors, their readings, device status, actuator status, manual controls, switch between auto/manual modes, and an emergency stop button.

The fifth goal was remote monitoring and practical implementation. It was attained by setting up a local server on the Raspberry Pi, static IP allocation, service management using Docker, wireless access to the Raspberry Pi via VNC, and remote access using a domain and Cloudflare if applicable.

7.4 Discussion of Final Outcomes

In the end, it turned out that implementing an inexpensive IoT-based hydroponics automation system is possible with the help of ESP32 microcontrollers and the Raspberry Pi 5 board. The independent ESP32 monitoring scheme was one of the crucial results of the work. In the solution, the average sensor readings were not calculated, but each of the ESP32 devices was shown independently on the dashboard.

This allowed improving fault detection since it would be easy to detect faulty readings, telemetry absence, or even the disconnected status for each particular ESP32 node. The choice of JSON-based MQTT messaging for sensor readings exchange offered a structured way to send data to the server and implement control decisions.

The results of the automation process have proved that water level management, nutrient administration, TDS control, drainage and refilling, overriding and emergency stops can be effectively executed. In addition to this, the coordination between valves and pumps was also enhanced by the fact that the inlet pump was controlled using the inlet valve, while the outlet pump used the outlet valve.

In general, this project illustrated the implementation of embedded technology, IoT connectivity, backend coding, dashboard creation, actuator control and remote monitoring in the context of hydroponics.

7.5 Limitations of the Project

Despite the success in reaching the desired objectives, certain practical issues arose during the development of the system. First, calibration of the pH and TDS sensors was crucial for obtaining accurate measurements. If calibration was done incorrectly, the decision-making concerning the doses of nutrients and the measurement of TDS could become problematic.

Second, the pH and TDS values took time to stabilize after the dosing or circulation of water occurred. When the nutrient dosing pump was turned on, there was a necessity for the solution to mix in order to obtain stabilized values. Thus, mixing delays had to be taken into account. Moreover, the system relied on Wi-Fi technology. If the signal got weak or the router was disconnected, the MQTT data transmission would experience certain delays. Remote monitoring, in turn, required a good internet connection and proper domain/Cloudflare configuration.

Another issue to consider was relay switching delays and power supply dependence. Stable voltage at 12 V was needed for pumps and solenoid valves. Furthermore, although the functional operation of the system was ensured, the period of time to test the growth of the crops was relatively short.

7.6 Recommendations for Improvement

Here are some recommendations that could help improve the system design going forward. First of all, high-quality industrial sensors could be implemented for pH, TDS measurements, as well as for other sensors involved in controlling plant growth. This will increase accuracy and reliability of sensor operation.

Secondly, data logging and graph history could be implemented in the dashboard, so users would be able to see how such parameters like pH, TDS, water level, humidity, and temperature have been changing over time. Finally, cloud storage could be implemented to store information collected from different sensors.

As for the mobile application, improvements could include push notification, authentication, and a better interface. In addition, automated A/B nutrient dosing can be implemented. Instead of a single nutrient dosing pump, separate pumps should be used, which will be turned on based on crop requirements.

Finally, as far as Raspberry Pi, ESP32 nodes, and crucial circuits of actuators are concerned, they should be equipped with solar backup power or UPS capability. Furthermore, since the system will operate in an environment where water may flow, waterproof housing needs to be installed for all sensors and cable wires.

Camera-based plant growth monitoring and AI-based decision support can be included in future work. The system will be able to monitor plant growth problems, evaluate the plant's health status, and provide control suggestions automatically with such systems.

7.7 Future Work

Further research might concentrate on increasing the efficiency, reliability, and intelligence of the system itself. It could include adding hydroponic channels and more nodes of ESP32. Such modifications will make the system ready for bigger cultivations and multi-crop farms.

Integration with greenhouse environment can be carried out, as well. One may add control of lights, air circulation, temperature, humidity, and carbon dioxide in the air. As a result, hydroponic farm will become a fully automated controlled environment agricultural setup.

Forecasting functionality can be introduced using the analysis of recorded data. This will allow predicting nutrient unbalance, changes in water level, or even sensor failure before affecting crops. Calibration features could be introduced to minimize manual intervention.

Notifications of any anomalies via mobile devices or emails could be made to include pH unbalance, insufficient water level, offline state of devices, pump malfunction, and errors in remote access. Long-term enhancements could be aimed at making the system ready for commercial use with industrial grade sensors and waterproof enclosure, power backup and cloud-based connection.

7.8 Final Summary

The project showed that an efficient and cost-effective IoT-based hydroponics automation system can indeed be developed using ESP32 nodes, Raspberry Pi 5, MQTT communication protocol, Server.js backend server, web dashboard, mobile interface compatibility, sensors, relays, solenoid valves, and pumps. The project succeeded in implementing a system with features such as real-time monitoring, automated water level control, nutrient addition based on pH value, drain-and-fill based on TDS level, manual intervention, emergency stop button, and remote monitoring.

The contribution made by Member 2 has been proved by implementing a number of features including the implementation of the local server Raspberry Pi 5, the MQTT communication using Mosquitto broker, the Server.js backend server, the dashboard communications, mobile application compatibility, network configuration, remote monitoring, and validation of the complete system.

REFERENCES

- [1] H. M. Resh, *Hydroponic Food Production*, 7th ed. Boca Raton, FL, USA: CRC Press, 2013.
- [2] D. Savvas and H. C. Passam, Eds., *Hydroponic Production of Vegetables and Ornamentals*. Athens, Greece: Embryo Publications, 2002.
- [3] A. Abu Sneineh and A. A. A. Shabaneh, "Design of a smart hydroponics monitoring system using an ESP32 microcontroller and the Internet of Things," *MethodsX*, vol. 11, p. 102401, 2023, doi: 10.1016/j.mex.2023.102401.
- [4] M. A. Awal, A. S. Pio, M. J. Mim, P. K. P. Partha, M. A. A. Kafi, and S. Farha, "A smart IoT-based hydroponics system for small-scale household in Bangladesh," *Smart Agricultural Technology*, vol. 12, p. 101163, 2025, doi: 10.1016/j.atech.2025.101163.
- [5] T. Baraskar, V. Khatri, P. Kolhe, M. Katyarmal, and S. Khedekar, "AI-powered automated hydroponic system for smart agriculture," *MethodsX*, vol. 15, p. 103579, 2025, doi: 10.1016/j.mex.2025.103579.
- [6] P. Rajak, A. Ganguly, S. Adhikary, and S. Bhattacharya, "Internet of Things and smart sensors in agriculture: Scopes and challenges," *Journal of Agriculture and Food Research*, vol. 14, p. 100776, 2023, doi: 10.1016/j.jafr.2023.100776.
- [7] E. Navarro, N. Costa, and A. Pereira, "A systematic review of IoT solutions for smart farming," *Sensors*, vol. 20, no. 15, p. 4231, 2020, doi: 10.3390/s20154231.
- [8] M. S. Farooq, S. Riaz, A. Abid, T. Umer, and Y. B. Zikria, "Role of IoT technology in agriculture: A systematic literature review," *Electronics*, vol. 9, no. 2, p. 319, 2020, doi: 10.3390/electronics9020319.
- [9] L. García, L. Parra, J. M. Jimenez, J. Lloret, and P. Lorenz, "IoT-based smart irrigation systems: An overview on the recent trends on sensors and IoT systems for irrigation in precision agriculture," *Sensors*, vol. 20, no. 4, p. 1042, 2020, doi: 10.3390/s20041042.
- [10] A. Soussi, E. Zero, R. Sacile, D. Trincherro, and M. Fossa, "Smart sensors and smart data for precision agriculture: A review," *Sensors*, vol. 24, no. 8, p. 2647, 2024, doi: 10.3390/s24082647.
- [11] S. Gupta, S. Chowdhury, R. Govindaraj, K. T. T. Amesho, S. Shangdiar, T. Kadhila, and S. Iikela, "Smart agriculture using IoT for automated irrigation, water and energy efficiency," *Smart Agricultural Technology*, vol. 12, p. 101081, 2025, doi: 10.1016/j.atech.2025.101081.
- [12] Espressif Systems, "ESP32 technical reference manual," Espressif Systems. Accessed: May 30, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

- [13] Espressif Systems, “ESP-IDF programming guide: ESP32,” Espressif Documentation. Accessed: May 30, 2026. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html>
- [14] Espressif Systems, “ESP-MQTT: ESP32,” Espressif Documentation. Accessed: May 30, 2026. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mqtt.html>
- [15] Raspberry Pi Ltd., “Raspberry Pi 5 product brief,” Raspberry Pi Documentation. Accessed: May 30, 2026. [Online]. Available: <https://pip.raspberrypi.com/documents/RP-008348-DS-raspberry-pi-5-product-brief.pdf>
- [16] Raspberry Pi Ltd., “Raspberry Pi computer hardware,” Raspberry Pi Documentation. Accessed: May 30, 2026. [Online]. Available: <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html>
- [17] OASIS, “MQTT version 5.0,” OASIS Standard. Accessed: May 30, 2026. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [18] Eclipse Foundation, “Eclipse Mosquitto: An open source MQTT broker,” Eclipse Mosquitto. Accessed: May 30, 2026. [Online]. Available: <https://mosquitto.org/>
- [19] Eclipse Foundation, “mosquitto.conf man page,” Eclipse Mosquitto Documentation. Accessed: May 30, 2026. [Online]. Available: <https://mosquitto.org/man/mosquitto-conf-5.html>
- [20] OpenJS Foundation, “Server.js documentation,” Server.js. Accessed: May 30, 2026. [Online]. Available: <https://nodejs.org/docs/latest/api/>
- [21] OpenJS Foundation, “Server.js — Run JavaScript everywhere,” Server.js. Accessed: May 30, 2026. [Online]. Available: <https://nodejs.org/en>
- [22] Socket.IO, “Socket.IO documentation,” Socket.IO. Accessed: May 30, 2026. [Online]. Available: <https://socket.io/>
- [23] Docker Inc., “Docker Compose documentation,” Docker Documentation. Accessed: May 30, 2026. [Online]. Available: <https://docs.docker.com/compose/>
- [24] Cloudflare, “Cloudflare Tunnel,” Cloudflare Documentation. Accessed: May 30, 2026. [Online]. Available: <https://developers.cloudflare.com/tunnel/>
- [25] DFRobot, “Gravity: Analog pH Sensor V2,” DFRobot Wiki. Accessed: May 30, 2026. [Online]. Available: <https://wiki.dfrobot.com/sen0161-v2/>
- Note:** Verify before submission that this is the exact pH sensor model used in the project.
- [26] DFRobot, “Gravity: Analog TDS Sensor/Meter for Arduino,” DFRobot Wiki. Accessed: May 30, 2026. [Online]. Available: <https://wiki.dfrobot.com/sen0244/>
- Note:** Verify before submission that this is the exact TDS sensor model used in the project.

[27] Aosong Electronics Co., Ltd., “DHT11 digital temperature and humidity sensor datasheet,” Aosong Electronics. Accessed: May 30, 2026. [Online]. Available: https://www.electronicoscaldas.com/datasheet/DHT11_Aosong.pdf

[28] ElecFreaks, “Ultrasonic ranging module HC-SR04 datasheet,” SparkFun Electronics. Accessed: May 30, 2026. [Online]. Available: <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>

[29] Ningbo Songle Relay Co., Ltd., “SRD-05VDC-SL-C relay datasheet,” Songle Relay. Accessed: May 30, 2026. [Online]. Available: <https://www.handsontec.com/dataspecs/relay/SRD-05VDC-SL-C.pdf>

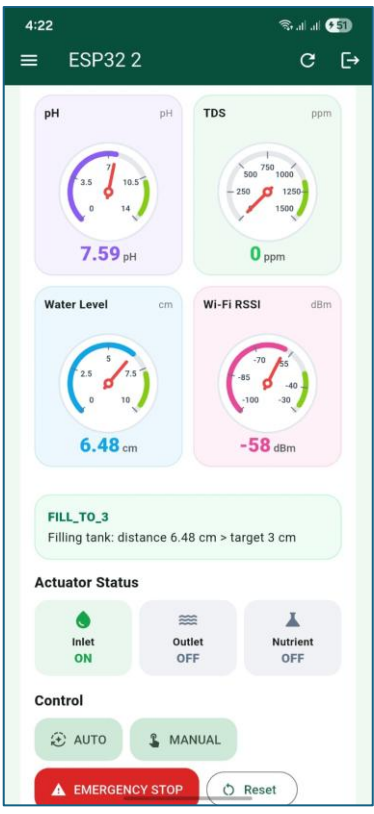
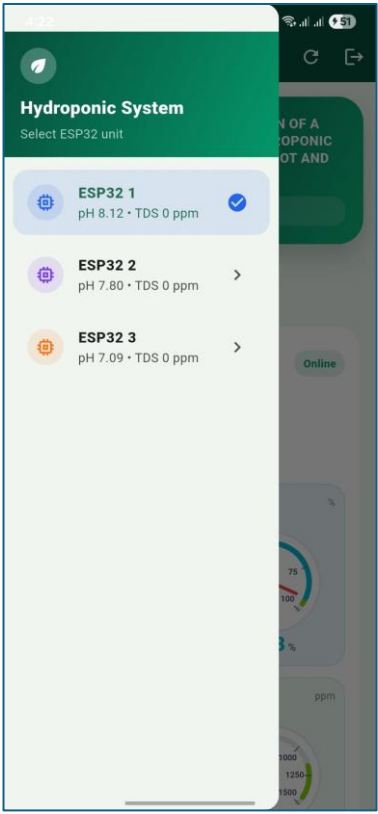
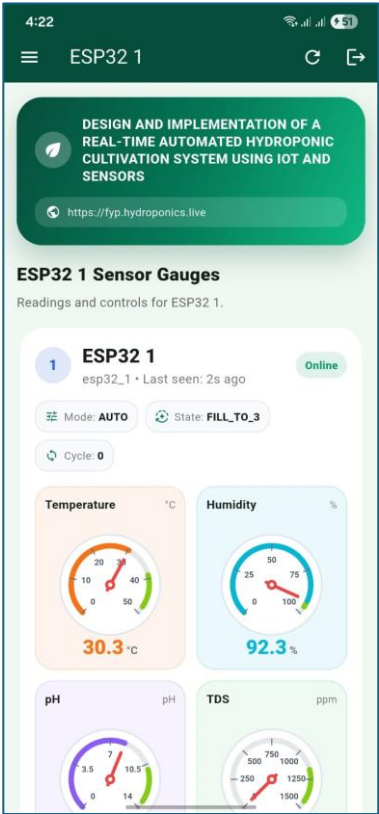
Note: Verify before submission that this relay model matches the relay module used in the project.

APPENDIX A

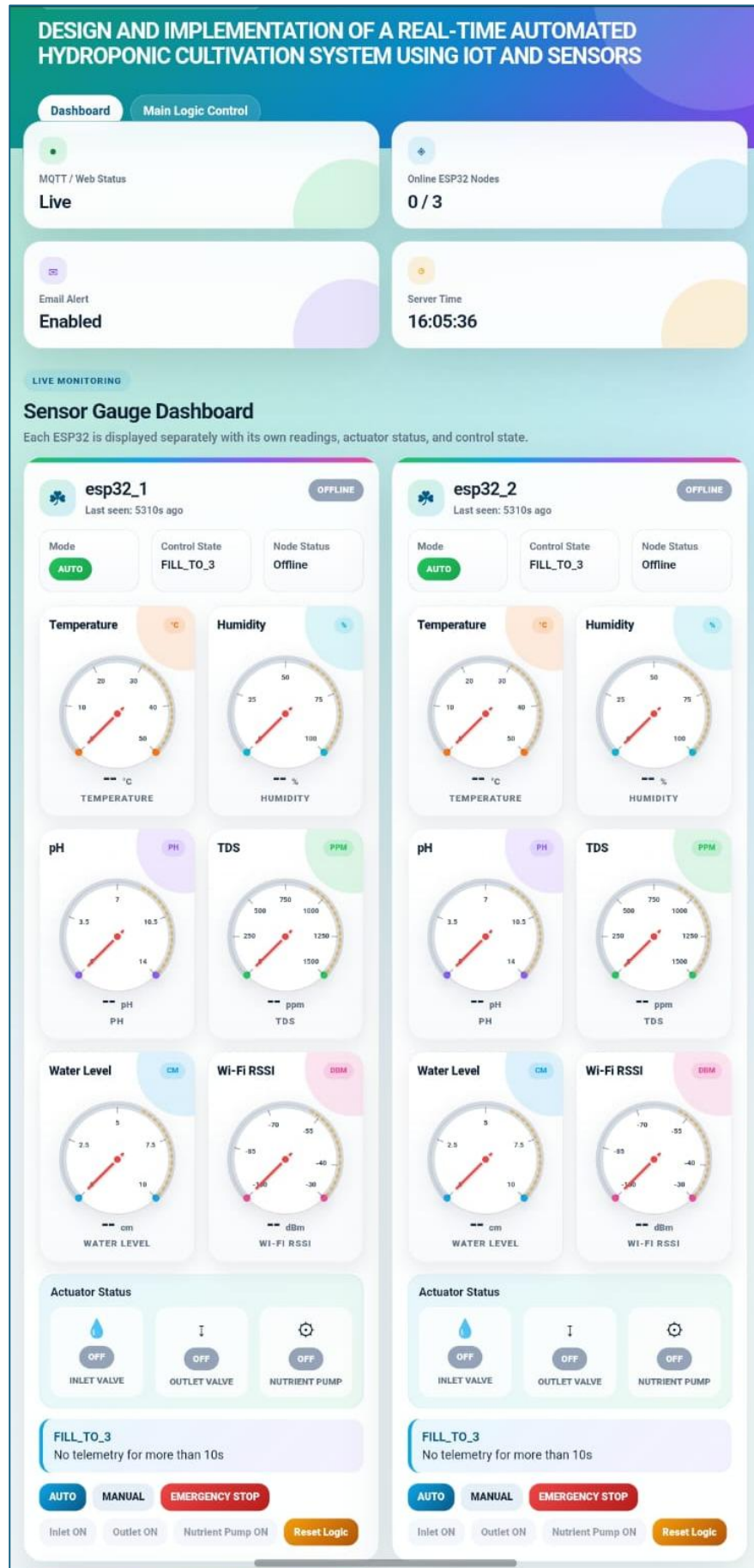
Overall System



Mobile Application



Web Dash Board



APPENDIX B

Controlling Code

```
const EventEmitter = require('events');
const config = require('./config');
const email = require('./email');

const ControllerState = Object.freeze({
  FILL_TO_3: 'FILL_TO_3',
  CHECK_PH: 'CHECK_PH',
  DOSE_NUTRIENT: 'DOSE_NUTRIENT',
  MIXING_WAIT: 'MIXING_WAIT',
  WAIT_TDS: 'WAIT_TDS',
  DRAIN_TO_7: 'DRAIN_TO_7',
  REFILL_TO_3: 'REFILL_TO_3',
  EMERGENCY_STOP: 'EMERGENCY_STOP',
  MANUAL: 'MANUAL',
  SENSOR_ERROR: 'SENSOR_ERROR',
});

function nowIso() {
  return new Date().toISOString();
}

function safeNumber(value) {
  const num = Number(value);
  return Number.isFinite(num) ? num : null;
}

function actuatorState(inlet = false, outlet = false, nutrientPump = false) {
  return {
    inlet: Boolean(inlet),
    outlet: Boolean(outlet),
    nutrientPump: Boolean(nutrientPump),
  };
}

function createEmptyDevice(deviceId) {
  return {
    deviceId,
    online: false,
    mode: 'auto',
    emergencyStop: false,
    lastSeen: null,
    telemetry: null,
    status: null,
    valves: actuatorState(false, false, false),
    controller: {
      state: ControllerState.FILL_TO_3,
      message: 'Waiting for telemetry',
    },
  };
}
```

```

        cycleCount: 0,
        nutrientDoseCount: 0,
        doseOffAt: null,
        mixWaitUntil: null,
        updatedAt: nowIso(),
    },
    alertFlags: {
        phHighActive: false,
        phEmailSending: false,
    },
    logs: [],
};
}

class HydroponicController extends EventEmitter {
    constructor({ mqttPublish }) {
        super();
        this.mqttPublish = mqttPublish;
        this.devices = new Map();

        config.deviceIds.forEach((deviceId) => {
            this.devices.set(deviceId, createEmptyDevice(deviceId));
        });

        setInterval(() => this.checkDeviceTimeouts(), 1000);
    }

    getOrCreateDevice(deviceId) {
        if (!this.devices.has(deviceId)) {
            this.devices.set(deviceId, createEmptyDevice(deviceId));
        }
        return this.devices.get(deviceId);
    }

    getPublicState() {
        return {
            thresholds: config.thresholds,
            emailEnabled: email.isEmailEnabled(),
            devices: Array.from(this.devices.values()),
            serverTime: nowIso(),
        };
    }

    addLog(deviceId, level, message, source = 'server') {
        const device = this.getOrCreateDevice(deviceId);
        const entry = {
            deviceId,
            level,
            message,
            source,

```

```

        time: nowIso(),
    };

    device.logs.unshift(entry);
    device.logs = device.logs.slice(0, 80);

    this.emit('log', entry);
    this.emit('state', this.getPublicState());
}

updateStatus(deviceId, statusPayload) {
    const device = this.getOrCreateDevice(deviceId);
    device.status = statusPayload;
    device.online = Boolean(statusPayload.online);

    if (device.online) {
        device.lastSeen = Date.now();
    }

    this.emit('state', this.getPublicState());
}

updateTelemetry(deviceId, telemetry) {
    const device = this.getOrCreateDevice(deviceId);

    device.telemetry = telemetry;
    device.lastSeen = Date.now();
    device.online = true;

    if (telemetry.valves) {
        device.valves = actuatorState(
            telemetry.valves.inlet,
            telemetry.valves.outlet,
            telemetry.valves.nutrient_pump ?? telemetry.valves.nutrientPump
        );
    }

    if (telemetry.emergency_stop === true) {
        device.emergencyStop = true;
    }

    this.runAutoLogic(device);
    this.emit('state', this.getPublicState());
}

handleEspLog(deviceId, logPayload) {
    this.addLog(
        deviceId,
        logPayload.level || 'INFO',
        logPayload.message || JSON.stringify(logPayload),
    );
}

```

```

        'esp32'
    );
}

setMode(deviceId, mode) {
    const device = this.getOrCreateDevice(deviceId);
    if (!['auto', 'manual'].includes(mode)) {
        throw new Error('Mode must be auto or manual');
    }

    device.mode = mode;

    if (mode === 'manual') {
        device.controller.state = ControllerState.MANUAL;
        device.controller.message = 'Manual mode: web quick controls are
enabled';
        device.controller.updatedAt = nowIso();
        this.sendActuatorCommand(deviceId, false, false, false, 'mode_manual');
    } else {
        device.emergencyStop = false;
        this.resetDoseTracker(device);
        device.controller.state = ControllerState.FILL_TO_3;
        device.controller.message = 'Auto mode restarted from fill state';
        device.controller.updatedAt = nowIso();
        this.sendEmergencyStop(deviceId, false, 'mode_auto_clear_stop');
    }

    this.addLog(deviceId, 'INFO', `Mode changed to ${mode}`, 'web');
    this.emit('state', this.getPublicState());
}

resetDoseTracker(device) {
    device.controller.nutrientDoseCount = 0;
    device.controller.doseOffAt = null;
    device.controller.mixWaitUntil = null;
}

resetController(deviceId) {
    const device = this.getOrCreateDevice(deviceId);
    device.controller.state = ControllerState.FILL_TO_3;
    device.controller.message = 'Controller reset to initial fill state';
    device.controller.updatedAt = nowIso();
    device.alertFlags.phHighActive = false;
    device.emergencyStop = false;
    this.resetDoseTracker(device);
    this.sendEmergencyStop(deviceId, false, 'reset_controller_clear_stop');
    this.sendActuatorCommand(deviceId, false, false, false,
'reset_controller');
    this.addLog(deviceId, 'INFO', 'Controller state reset', 'web');
}

```

```

setEmergencyStop(deviceId, stop) {
  const device = this.getOrCreateDevice(deviceId);
  device.emergencyStop = Boolean(stop);

  if (device.emergencyStop) {
    device.controller.state = ControllerState.EMERGENCY_STOP;
    device.controller.message = 'Emergency stop active: inlet, outlet, and
nutrient pump are forced OFF';
    this.resetDoseTracker(device);
    this.sendEmergencyStop(deviceId, true, 'web');
  } else {
    device.controller.state = device.mode === 'auto' ?
ControllerState.FILL_TO_3 : ControllerState.MANUAL;
    device.controller.message = device.mode === 'auto'
? 'Emergency stop cleared: auto mode restarted'
: 'Emergency stop cleared: manual mode active';
    this.sendEmergencyStop(deviceId, false, 'web');
  }

  device.controller.updatedAt = nowIso();
  this.addLog(deviceId, stop ? 'WARN' : 'INFO', stop ? 'Emergency stop
enabled' : 'Emergency stop cleared', 'web');
}

manualValveControl(deviceId, actuator, on) {
  const device = this.getOrCreateDevice(deviceId);

  if (device.emergencyStop) {
    throw new Error('Emergency stop is active. Clear emergency stop
first.');
```

```

        throw new Error('Safety rule: nutrient pump can run only when inlet and
outlet are OFF.');
```

```

    }

    this.sendActuatorCommand(deviceId, inlet, outlet, nutrientPump,
'web_manual');
    this.addLog(
        deviceId,
        'INFO',
        `Manual actuator command: inlet=${inlet ? 'ON' : 'OFF'}, outlet=${outlet
? 'ON' : 'OFF'}, nutrient_pump=${nutrientPump ? 'ON' : 'OFF'}`,
        'web'
    );
}

sendValveCommand(deviceId, inlet, outlet, source) {
    this.sendActuatorCommand(deviceId, inlet, outlet, false, source);
}

sendActuatorCommand(deviceId, inlet, outlet, nutrientPump, source) {
    let safeInlet = Boolean(inlet);
    let safeOutlet = Boolean(outlet);
    let safePump = Boolean(nutrientPump);

    // Safety: never allow both water valves ON together.
    if (safeInlet && safeOutlet) {
        safeInlet = false;
        safeOutlet = false;
    }

    // Safety: nutrient dosing must not happen while filling or draining.
    if (safePump && (safeInlet || safeOutlet)) {
        safePump = false;
    }

    const topic = `hydroponics/${deviceId}/command`;
    const payload = {
        type: 'set_actuators',
        source,
        inlet: safeInlet,
        outlet: safeOutlet,
        nutrient_pump: safePump,
        at: nowIso(),
    };

    this.mqttPublish(topic, payload);

    const device = this.getOrCreateDevice(deviceId);
    device.valves = actuatorState(safeInlet, safeOutlet, safePump);
}

```

```

sendEmergencyStop(deviceId, stop, source) {
  const topic = `hydroponics/${deviceId}/command`;
  const payload = {
    type: 'emergency_stop',
    source,
    stop: Boolean(stop),
    at: nowIso(),
  };

  this.mqttPublish(topic, payload);

  if (stop) {
    const device = this.getOrCreateDevice(deviceId);
    device.valves = actuatorState(false, false, false);
  }
}

publishControllerState(device) {
  const topic = `hydroponics/${device.deviceId}/controller`;
  this.mqttPublish(topic, {
    device_id: device.deviceId,
    mode: device.mode,
    emergency_stop: device.emergencyStop,
    controller: device.controller,
    valves: device.valves,
    thresholds: config.thresholds,
    at: nowIso(),
  }, true);
}

setControllerState(device, state, message) {
  if (device.controller.state !== state || device.controller.message !==
message) {
    device.controller.state = state;
    device.controller.message = message;
    device.controller.updatedAt = nowIso();
    this.addLog(device.deviceId, 'INFO', `${state}: ${message}`,
'controller');
  }

  this.publishControllerState(device);
}

async maybeSendPhAlert(device, ph) {
  const threshold = config.thresholds.phHighAlert;

  if (ph > threshold && !device.alertFlags.phHighActive &&
!device.alertFlags.phEmailSending) {
    device.alertFlags.phHighActive = true;
  }
}

```

```

        device.alertFlags.phEmailSending = true;
        this.addLog(device.deviceId, 'WARN', `High pH detected: ${ph.toFixed(2)}
> ${threshold}; nutrient pump dosing will be attempted`, 'controller');

        try {
            const result = await email.sendPhHighAlert({ deviceId:
device.deviceId, ph: ph.toFixed(2), threshold });
            this.addLog(device.deviceId, result.sent ? 'INFO' : 'WARN',
result.sent ? 'High pH email alert sent' : `High pH email not sent:
${result.reason}`, 'controller');
        } catch (err) {
            this.addLog(device.deviceId, 'ERROR', `Failed to send high pH email:
${err.message}`, 'controller');
        } finally {
            device.alertFlags.phEmailSending = false;
        }
    }

    if (ph <= threshold) {
        device.alertFlags.phHighActive = false;
    }
}

runAutoLogic(device) {
    const readings = device.telemetry?.readings || {};
    const distance = safeNumber(readings.distance_cm);
    const ph = safeNumber(readings.ph);
    const tds = safeNumber(readings.tds_ppm);
    const th = config.thresholds;
    const now = Date.now();

    if (device.emergencyStop) {
        this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_emergency_stop');
        this.setControllerState(device, ControllerState.EMERGENCY_STOP,
'Emergency stop active: inlet, outlet, and nutrient pump OFF');
        return;
    }

    if (device.mode !== 'auto') {
        this.setControllerState(device, ControllerState.MANUAL, 'Manual mode
active: auto control paused');
        return;
    }

    if (distance === null) {
        this.sendActuatorCommand(device.deviceId, false, false, false,
'sensor_error');
        this.setControllerState(device, ControllerState.SENSOR_ERROR,
'Ultrasonic distance missing: all actuators OFF for safety');
    }
}

```

```

    return;
}

switch (device.controller.state) {
  case ControllerState.FILL_TO_3:
  case ControllerState.SENSOR_ERROR:
  case ControllerState.EMERGENCY_STOP:
  case ControllerState.MANUAL: {
    this.resetDoseTracker(device);
    if (distance > th.targetDistanceCm) {
      this.sendActuatorCommand(device.deviceId, true, false, false,
'auto_fill_to_3');
      this.setControllerState(device, ControllerState.FILL_TO_3, `Filling
tank: distance ${distance.toFixed(2)} cm > target ${th.targetDistanceCm} cm`);
    } else {
      this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_fill_reached');
      this.setControllerState(device, ControllerState.CHECK_PH, `Water
level reached target: ${distance.toFixed(2)} cm`);
    }
    break;
  }

  case ControllerState.CHECK_PH: {
    this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_check_ph');

    if (ph === null) {
      this.setControllerState(device, ControllerState.CHECK_PH, 'Waiting
for valid pH reading');
      break;
    }

    this.maybeSendPhAlert(device, ph);

    if (ph > th.phHighAlert) {
      if (device.controller.nutrientDoseCount >= th.nutrientMaxDoses) {
        this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_max_nutrient_doses');
        this.setControllerState(device, ControllerState.CHECK_PH, `pH HIGH
${ph.toFixed(2)} > ${th.phHighAlert}; max nutrient doses reached. Manual
correction required.`);
      } else {
        device.controller.nutrientDoseCount += 1;
        device.controller.doseOffAt = now + th.nutrientDoseMs;
        device.controller.mixWaitUntil = null;
        this.sendActuatorCommand(device.deviceId, false, false, true,
'auto_nutrient_dose');
        this.setControllerState(device, ControllerState.DOSE_NUTRIENT, `pH
HIGH ${ph.toFixed(2)} > ${th.phHighAlert}; nutrient pump ON for

```

```

    ${th.nutrientDoseMs / 1000}s (dose
    ${device.controller.nutrientDoseCount}/${th.nutrientMaxDoses}))`);
    }
    } else {
        this.resetDoseTracker(device);
        this.setControllerState(device, ControllerState.WAIT_TDS, `pH
stable: ${ph.toFixed(2)} <= ${th.phHighAlert}`);
    }
    break;
}

    case ControllerState.DOSE_NUTRIENT: {
        if (now < Number(device.controller.doseOffAt || 0)) {
            const remaining = Math.ceil((Number(device.controller.doseOffAt) -
now) / 1000);
            this.sendActuatorCommand(device.deviceId, false, false, true,
'auto_nutrient_dose_hold');
            this.setControllerState(device, ControllerState.DOSE_NUTRIENT,
`Nutrient pump dosing in progress. Remaining about ${remaining}s`);
        } else {
            device.controller.mixWaitUntil = now + th.nutrientMixMs;
            device.controller.doseOffAt = null;
            this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_nutrient_dose_complete');
            this.setControllerState(device, ControllerState.MIXING_WAIT,
`Nutrient pump OFF. Mixing/waiting ${th.nutrientMixMs / 1000}s before pH
recheck`);
        }
        break;
    }

    case ControllerState.MIXING_WAIT: {
        this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_mixing_wait');
        const waitUntil = Number(device.controller.mixWaitUntil || 0);
        if (now < waitUntil) {
            const remaining = Math.ceil((waitUntil - now) / 1000);
            this.setControllerState(device, ControllerState.MIXING_WAIT, `Mixing
wait active. Rechecking pH in about ${remaining}s`);
        } else {
            device.controller.mixWaitUntil = null;
            this.setControllerState(device, ControllerState.CHECK_PH, 'Mixing
wait complete. Rechecking pH');
        }
        break;
    }

    case ControllerState.WAIT_TDS: {
        this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_wait_tds');
    }

```

```

        if (tds === null) {
            this.setControllerState(device, ControllerState.WAIT_TDS, 'Waiting
for valid TDS reading');
            break;
        }

        if (tds >= th.tdsTargetPpm) {
            this.sendActuatorCommand(device.deviceId, false, true, false,
'auto_tds_reached_drain');
            this.setControllerState(device, ControllerState.DRAIN_TO_7, `TDS
reached ${tds.toFixed(0)} ppm; draining until ${th.drainStopCm} cm`);
        } else {
            this.setControllerState(device, ControllerState.WAIT_TDS, `Waiting
for TDS: ${tds.toFixed(0)} ppm < ${th.tdsTargetPpm} ppm`);
        }
        break;
    }

    case ControllerState.DRAIN_TO_7: {
        if (distance < th.drainStopCm) {
            this.sendActuatorCommand(device.deviceId, false, true, false,
'auto_drain_to_7');
            const nearText = distance >= th.drainNearCm ? 'near low level' :
'draining';
            this.setControllerState(device, ControllerState.DRAIN_TO_7,
`${nearText}: distance ${distance.toFixed(2)} cm < stop ${th.drainStopCm}
cm`);
        } else {
            this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_drain_complete');
            this.setControllerState(device, ControllerState.REFILL_TO_3, `Drain
complete: ${distance.toFixed(2)} cm >= ${th.drainStopCm} cm`);
        }
        break;
    }

    case ControllerState.REFILL_TO_3: {
        if (distance > th.targetDistanceCm) {
            this.sendActuatorCommand(device.deviceId, true, false, false,
'auto_refill_to_3');
            this.setControllerState(device, ControllerState.REFILL_TO_3,
`Refilling: distance ${distance.toFixed(2)} cm > target ${th.targetDistanceCm}
cm`);
        } else {
            this.sendActuatorCommand(device.deviceId, false, false, false,
'auto_cycle_complete');
            device.controller.cycleCount += 1;
            this.resetDoseTracker(device);
        }
    }

```

```

        this.setControllerState(device, ControllerState.CHECK_PH, `Cycle
${device.controller.cycleCount} complete; water level back to
${distance.toFixed(2)} cm`);
    }
    break;
}

default: {
    device.controller.state = ControllerState.FILL_TO_3;
    device.controller.message = 'Unknown state corrected to FILL_TO_3';
    this.resetDoseTracker(device);
    break;
}
}
}

checkDeviceTimeouts() {
    const timeout = config.thresholds.deviceTimeoutMs;
    const now = Date.now();
    let changed = false;

    for (const device of this.devices.values()) {
        if (device.lastSeen && now - device.lastSeen > timeout && device.online)
        {
            device.online = false;
            device.valves = actuatorState(false, false, false);
            device.controller.message = `No telemetry for more than ${timeout /
1000}s`;
            this.addLog(device.deviceId, 'WARN', 'Device telemetry timeout: marked
offline and local state set OFF', 'server');
            changed = true;
        }
    }

    if (changed) {
        this.emit('state', this.getPublicState());
    }
}

module.exports = {
    HydroponicController,
    ControllerState,
};

```